Lecture8 微服务运维



于晗

han_yu@sjtu.edu.cn

目录

- 1 微服务容器部署（Docker）
- 2 微服务容器编排（Kubernetes）
- 3 微服务系统运维（Devops）
- 4 小结

1.1 为什么需要容器化部署

- 部署问题

(1) 开发和生产环境或配置不同，导致开发环境运行良好的交付产物在生产环境出问题。

(2) 重复劳动：线上N台机器，每台都需要部署一次。

- 解决方案
  - 虚拟机：虚拟一套硬件并运行一套完整操作系统，然后在系统上运行所需应用
    - 缺点：

      (1) 资源占用多：虚机启动需要占用几百M的内存。

      (2) 冗余步骤多：系统级别的操作步骤，往往无法跳过，比如用户登录。

      (3) 启动慢：系统级的启动。

1.1 为什么需要容器化部署

- 解决方案
  - 容器：将应用程序及所有依赖打包在一起，提供隔离的运行环境，使应用程序可以在独立的文件系统、网络、进程空间等独立运行环境中运行。
    - 优点：

      （1）可移植性：应用与底层运行环境进行解耦，可快速移植和运行。

      （2）高效性：进程级别，共享操作系统和硬件资源，占用资源更少，启动停止更快速。

      （3）环境隔离：独立的文件系统、网络和进程空间，保证应用程序安全和稳定。

      （4）可伸缩性：通过容器编排工具，可以自动管理和调度容器的部署和扩展，提高了应用程序的可伸缩性和可靠性。

Docker就是容器技术的代表

1.2 Docker核心概念

- **镜像（Image）**

镜像可以看作是一个特殊的文件系统，除了提供容器运行时所需的程序、库、资源、配置等文件外，还包含了一些为运行时准备的一些配置参数（如匿名卷、环境变量、用户等）。

- **容器（Container）**

镜像（Image）和容器（Container）的关系，就像是面向对象程序设计中的类和实例一样，镜像是静态的定义，容器是镜像运行时的实体。容器可以被创建、启动、停止、删除、暂停等。

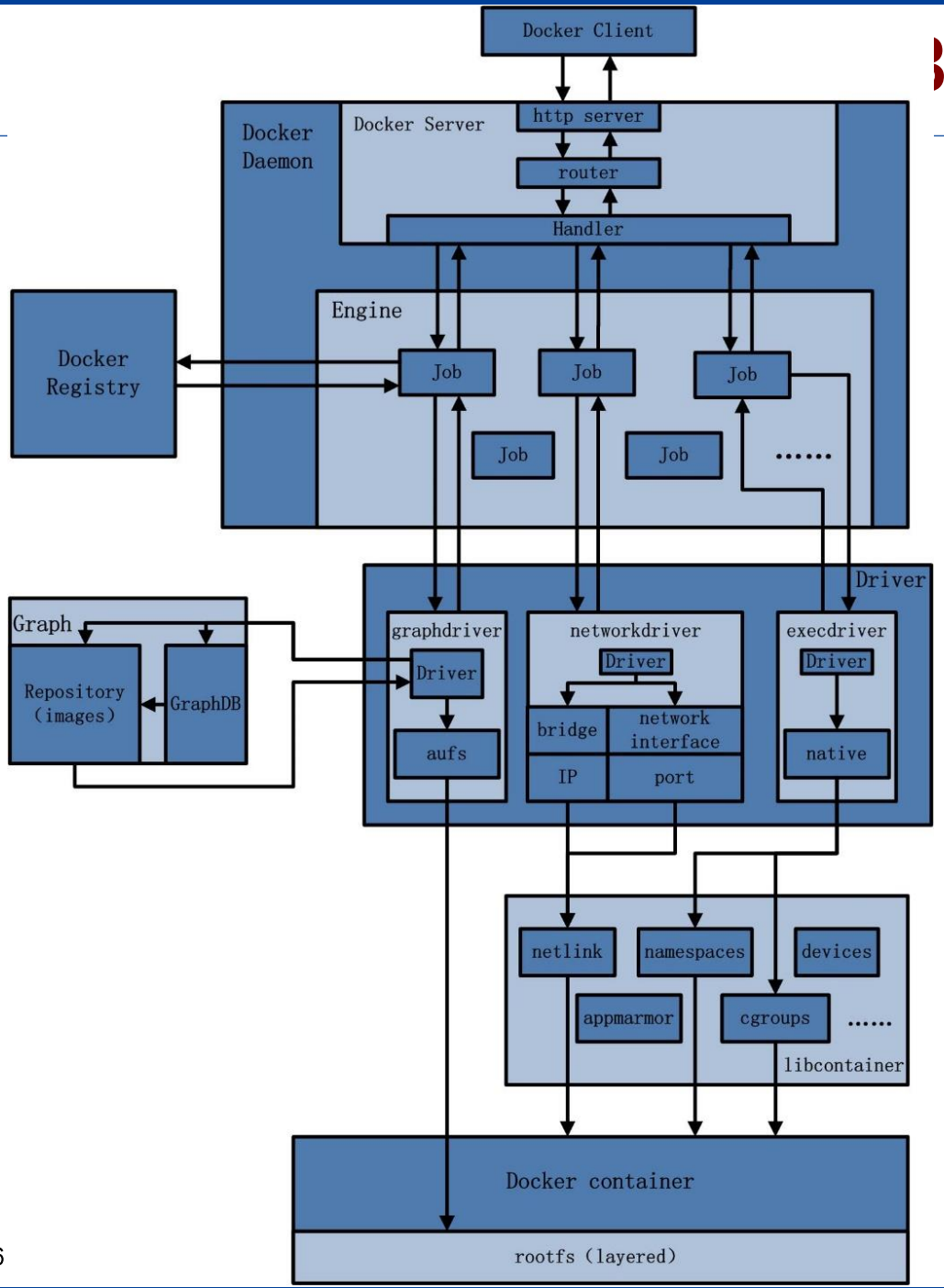
- Dockerfile

包含了一系列的命令和参数，用于自动化构建Docker镜像。Dockerfile使用特定的语法和格式来定义镜像的构建过程，通过读取Dockerfile中的内容，Docker可以自动地执行相应的步骤来构建镜像。

- **仓库（Repository）**

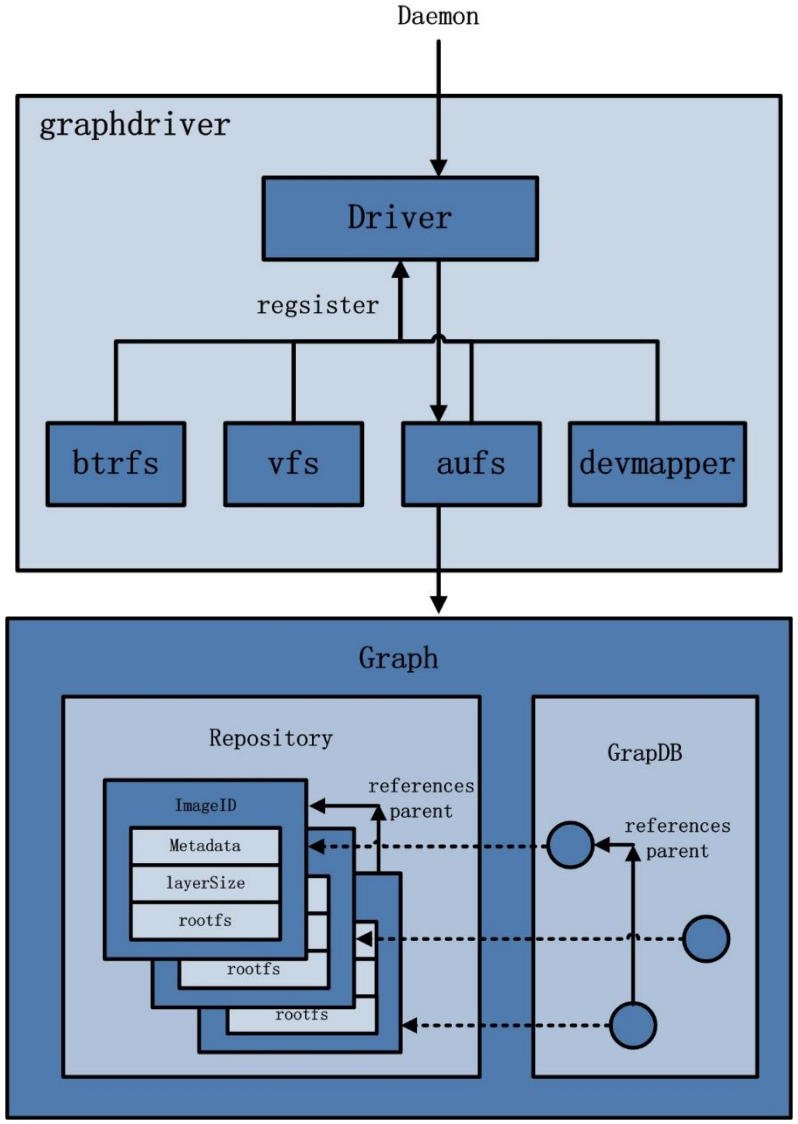
仓库可看成一个代码控制中心，用来集中存放镜像。

3 Docker的实现架构



- 用户使用Docker Client与Docker Daemon通信请求。
- Docker Daemon作为Docker架构中的主体部分：
  - Server：接收Client的请求。
  - Engine：执行Docker内部一系列工作（Job）。
- Job分类：
  - 拉取镜像：从Docker Registry中下载镜像，通过graphdriver将下载镜像以Graph形式存储。
  - 创建网络环境：通过networkdriver创建并配置Docker容器网络环境。
  - 限制运行资源/执行用户指令：通过execdriver来完成。
- libcontainer：独立的容器管理包，networkdriver以及execdriver都是通过libcontainer来实现具体对容器进行的操作。

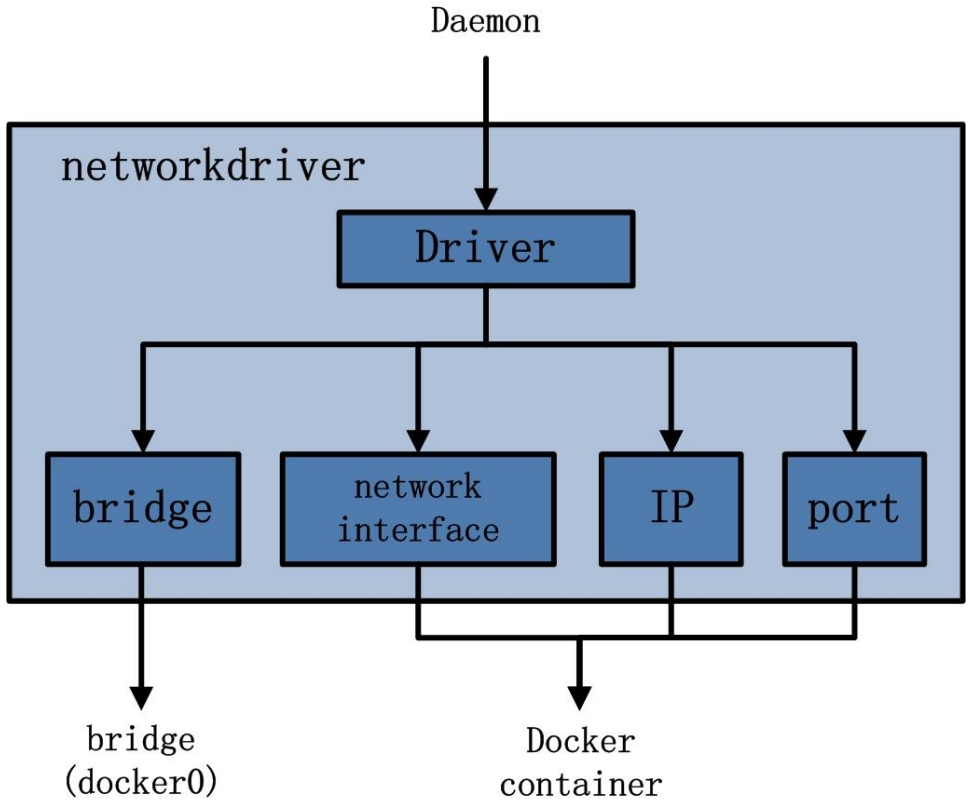
1.3 Docker的实现架构



- Graphdriver：主要用于完成容器镜像的管理，包括存储与获取。
  - 存储：docker pull下载的镜像由graphdriver存储到本地的指定目录（Graph）。
  - 获取：docker run用镜像来创建容器的时候由graphdriver到本地Graph中获取镜像。
- Graph
  - Repository：某类镜像的仓库，同一个repository内的镜像用tag来区分版本；具体镜像的存储信息包括该容器镜像的元数据、容器镜像的大小信息以及该容器镜像所代表的具体rootfs。
  - GraphDB：已下载容器镜像之间关系的记录者；构建在SQLite上的小型图数据库。

1.3 Docker的实现架构

➢ networkdriver

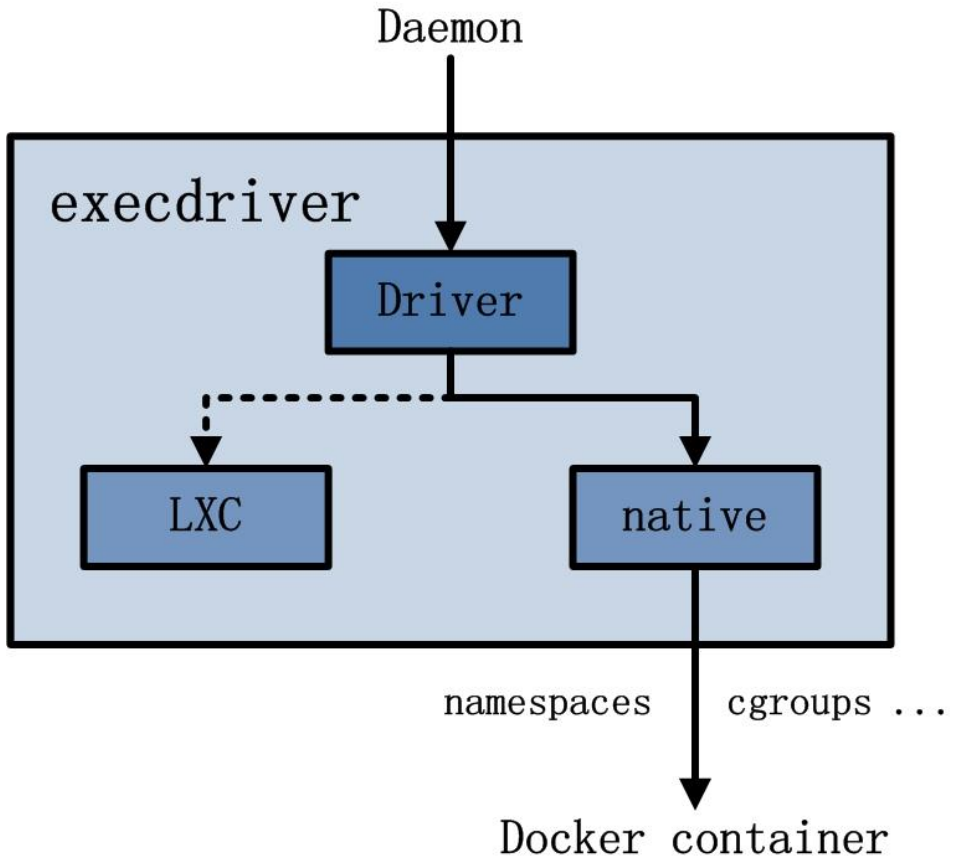


➢ networkdriver的用途是完成Docker容器网络环境的配置，其中包括：

- Docker启动时为Docker环境创建网桥。
- Docker容器创建时为其创建专属虚拟网卡设备。
- Docker容器分配IP、端口并与宿主机做端口映射，设置容器防火墙策略等。

1.3 Docker的实现架构

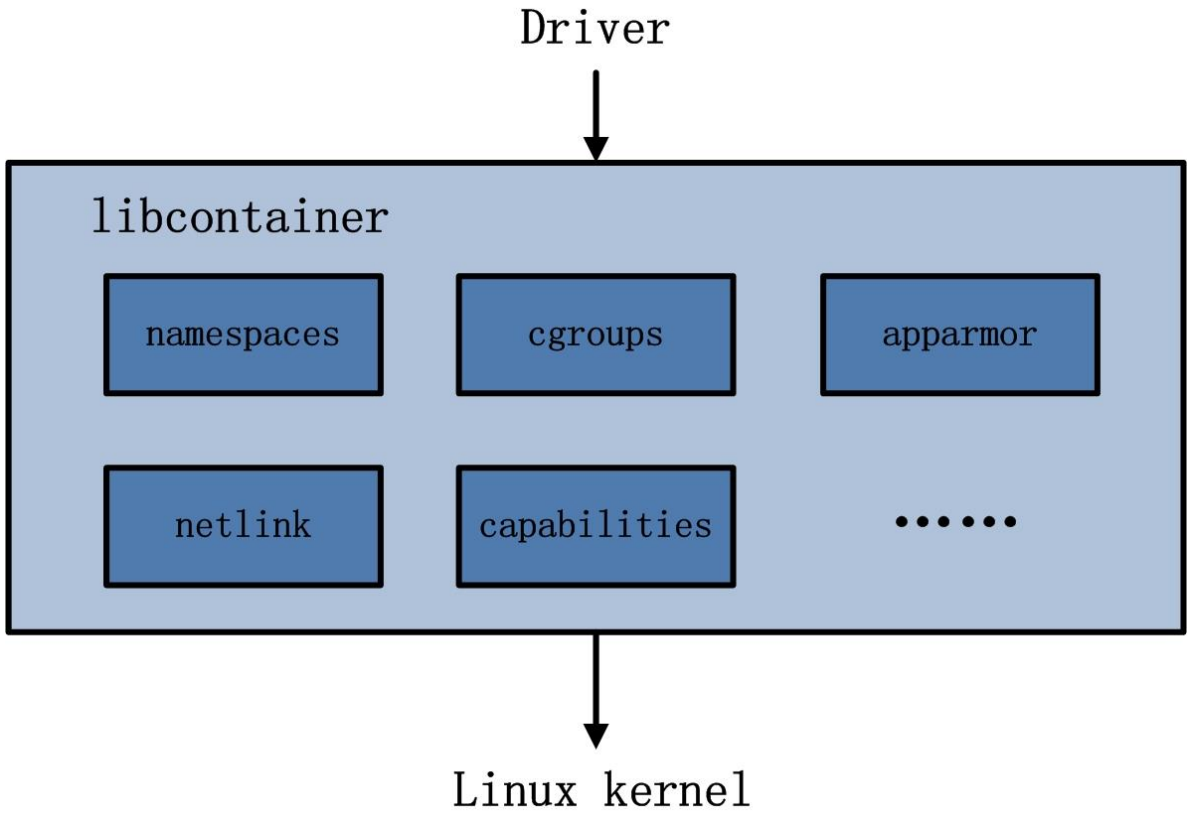
➢ execdriver



- execdriver是Docker容器的执行驱动，负责：
  - 创建容器运行命名空间。
  - 容器资源使用的统计与限制。
  - 容器内部进程的真正运行。

1.3 Docker的实现架构

➢ libcontainer



- libcontainer是Docker架构中一个使用Go语言设计实现的库：
  - Docker可以直接调用libcontainer，而最终操纵容器的namespace、cgroups、apparmor、网络设备以及防火墙规则等
  - libcontainer提供了一整套标准的接口来满足上层对容器管理的需求，屏蔽了Docker上层对容器的直接管理。

1.6 其他容器技术

OpenVZ

基于Linux内核的操作系统级虚拟化技术，允许多个隔离的Linux容器，共享同一宿主机的操作系统内核，每个容器有自己的文件系统、进程空间、用户ID和网络栈，但共享宿主机物理资源，与Docker差异：

- 隔离级别更深：Docker在应用层面，OpenVZ在操作系统层面。
- 由于在操作系统层面进行隔离，具有更高的安全性和稳定性。
- 主要基于Linux，可支持的操作系统有限。
- 管理和部署的复杂性更高。

Podman

开源的容器引擎，专为开发、运行和管理Linux OCI容器和容器映像而设计，与Docker差异：

- 不依赖守护进程，使用运行时进程直接与Linux内核和注册表交互。
- 不需要root访问权限，减少了潜在的安全风险。
- 不仅支持容器运行，还支持Pod运行，因此可以轻松迁移到k8s中。

目录

- 1 微服务容器部署（Docker）
- 2 微服务容器编排（Kubernetes）
- 3 微服务系统运维（Devops）
- 4 小结

2.1 为什么需要容器编排

- 考虑如下问题：

  在上面的Docker实践案例中，我们容器化部署了一个简单的Python后端应用，但在实际的场景下，对于一个应用往往还有前端、数据库等其他组件，考虑到性能和容错，这些组件往往不是单体架构，此时，该如何进行部署呢？

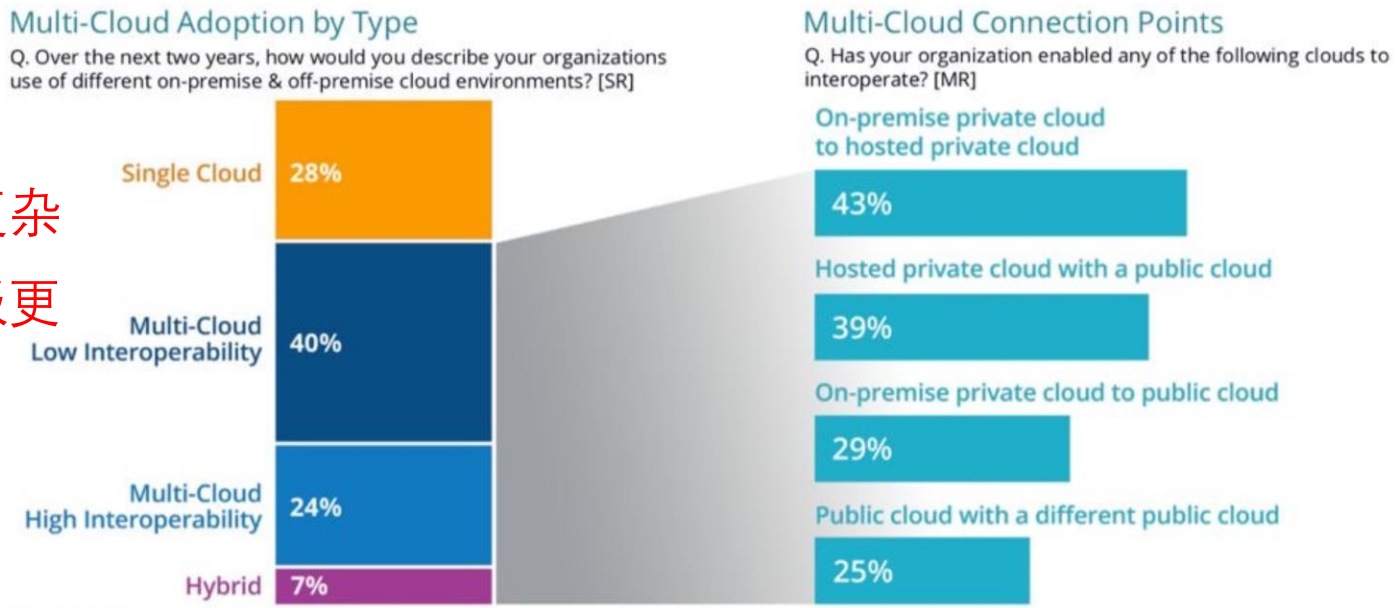
- 解决方案：
  - 手动部署多个Docker：

  （1）容器的数量愈来愈多，应用越来越复杂

  （2）传统插件、脚本部署方式不利于升级更新/回滚等操作

  （3）容器应用的服务发现与负载均衡

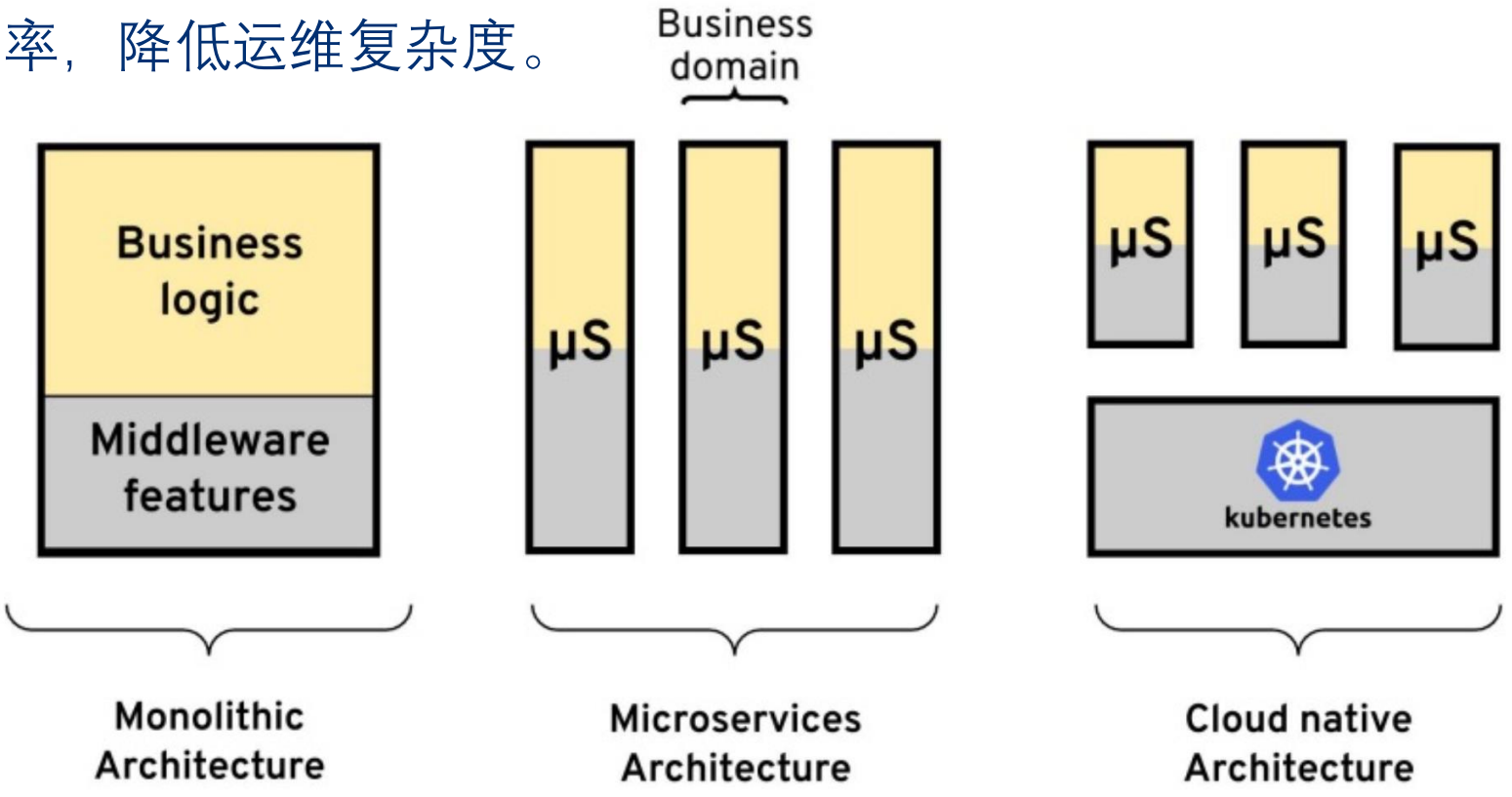
  （4）服务的持续可用性：自动修复



2.1 为什么需要容器编排

云原生

一种基于云计算的软件开发和部署方法论，它强调将应用程序和服务设计为云环境下的原生应用，以实现高可用性、可扩展性和灵活性。云原生的核心理念包括容器化、微服务、DevOps等，旨在帮助企业构建弹性可靠、松耦合、易管理、可观测的应用系统，提升交付效率，降低运维复杂度。



2.1 为什么需要容器编排

- Kubernetes

是编排容器的工具更是管理应用全生命周期的工具，从创建应用到应用的部署、服务发现、复杂均衡、弹性伸缩、自动修复等，都能提供极大便利。

K8s设计理念与云原生相契合，可看作云原生的一种实现方式和关键工具。它提供了强大的容器编排能力，使得开发者可以更加便捷地部署和管理云原生应用程序。K8s的目标是让部署容器化的应用简单并且高效，它提供了应用部署、规划、更新、维护的一种机制。通过K8s，开发者可以轻松地创建、扩展和管理容器化应用，依托于容器编排工具与微服务的组合，应用拥有了自动恢复能力、容错能力、故障隔离能力，让应用时刻处于可用的状态，从而充分发挥云平台的弹性和分布式优势。

2.2 Kubernetes核心概念

➢ API对象

用于描述和定义K8s中的各种资源和组件，k8s集群中的管理操作单元，例如：Pod、Service、Namespace等。

- 属性
  - 元数据（metadata）：uid，name，namespace，labels...
  - 规范（spec）：用户期望K8s 集群中系统达到的理想状态(Desired State)
  - 状态（status）：系统当前达到的实际状态

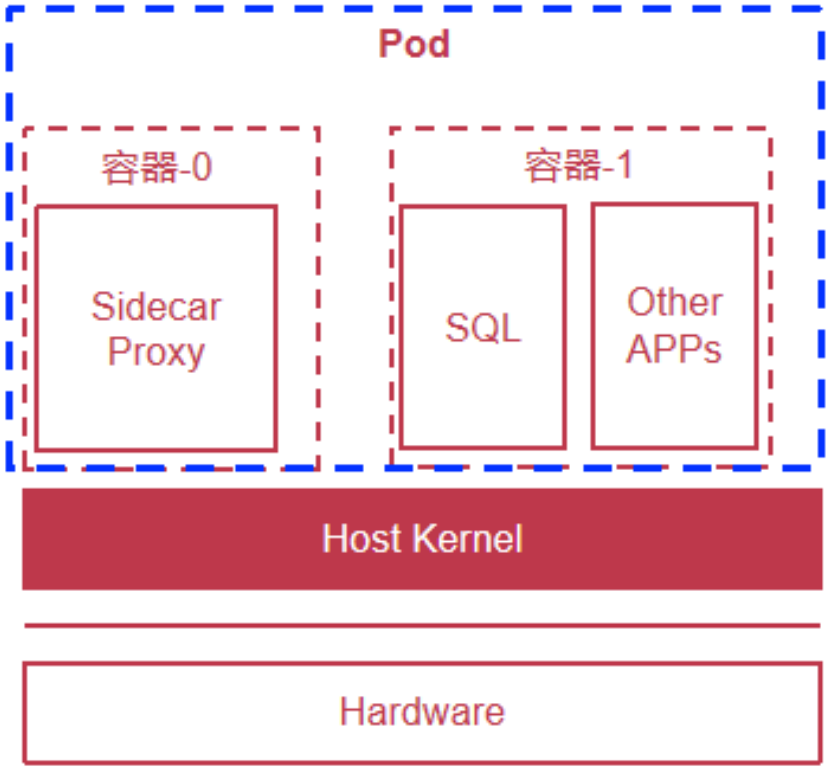
2.2 Kubernetes核心概念

API对象

- Pod

  Kubernetes 中创建和管理的、最小的可部署的计算单元

  - 容器抽象的扩展，由一个或多个容器组成
  - 有些应用程序需要支持同时存在的辅助进程，例如记录应用的日志或者监控数据的进程：Proxy
  - 可以天然共享网络和存储



2.2 Kubernetes核心概念

API对象

Service

K8s中一个抽象的概念，用于定义一组Pod的访问策略——通过标签选择器来选择一组Pod，并为这组Pod提供稳定的网络访问地址，而无需关心这些Pod的具体位置。具体有四种类型：

- ClusterIP：集群内部使用的IP地址，只能够在集群内部访问。
- NodePort：每个节点上开放一个特定的端口，通过该端口可以从集群外部访问Service。
- LoadBalancer：在云平台上创建一个负载均衡器，将外部流量转发到Service。
- ExternalName：Service映射到一个外部DNS名称，不直接提供负载均衡或代理功能。

NameSpace

Kubernetes集群中的虚拟集群，用于将集群资源隔离成不同的组，每个组有自己的资源配额和访问控制策略。通过Namespace，可以将不同的用户、团队或项目隔离开来，以避免资源竞争和冲突。默认情况下，Kubernetes集群会创建几个Namespace，如default、kube-system、kube-public。用户也可以根据需要创建自定义的Namespace。在Namespace内部，Pod、Service等资源的名称必须是唯一的，但在不同的Namespace之间，资源名称可以重复。

2.2 Kubernetes核心概念

基于期望状态的集群管理

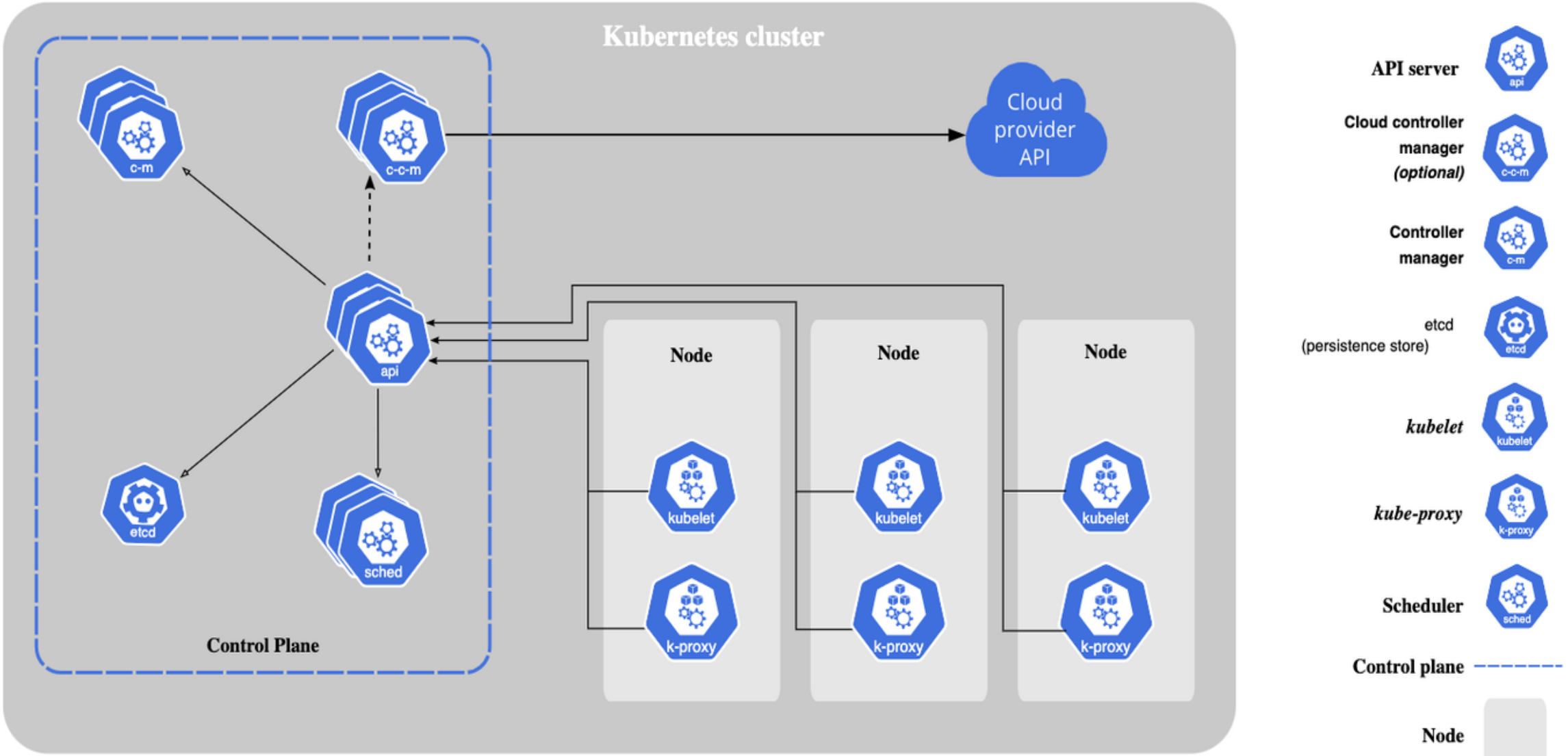
如果用户期望（Spec）Pod的副本数为3，而当前实际（Status）Pod的副本数为1，那么k8s将会尝试对集群进行更改，来满足期望状态：

- 副本数量
- 要完成的任务
- 预期的输入参数

核心是一种声明式命令

- 描述目标状态，而非具体操作
- 配置文件中是最终的对象和它们的状态
- 用户并不关心k8s怎么实现状态的匹配

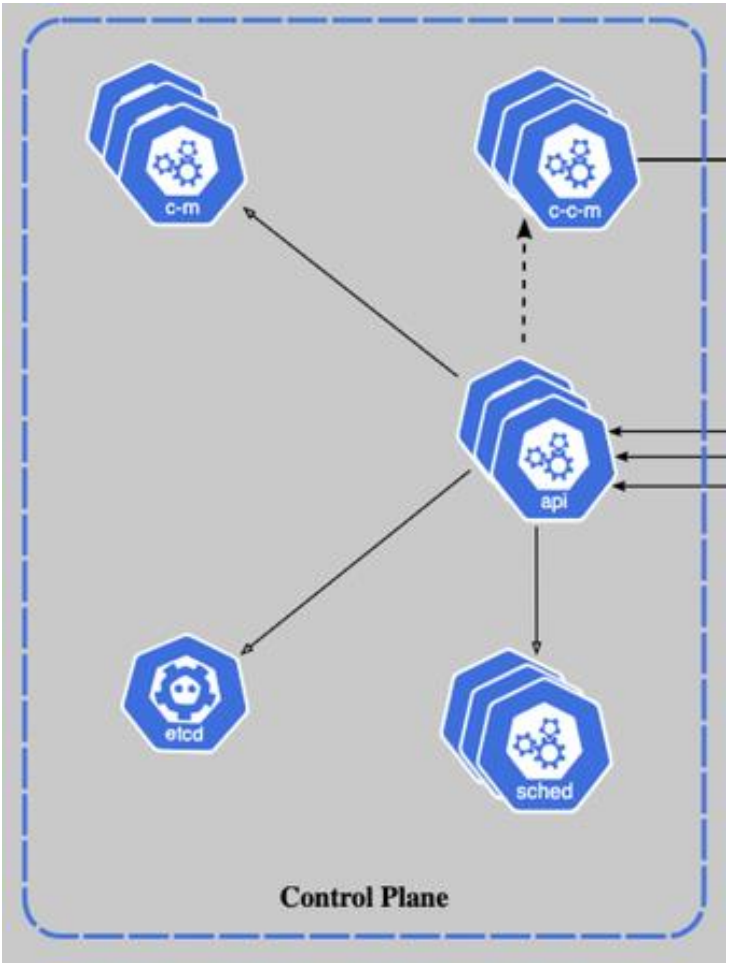
2.3 Kubernetes架构图



2.3 Kubernetes架构图

Control Plane

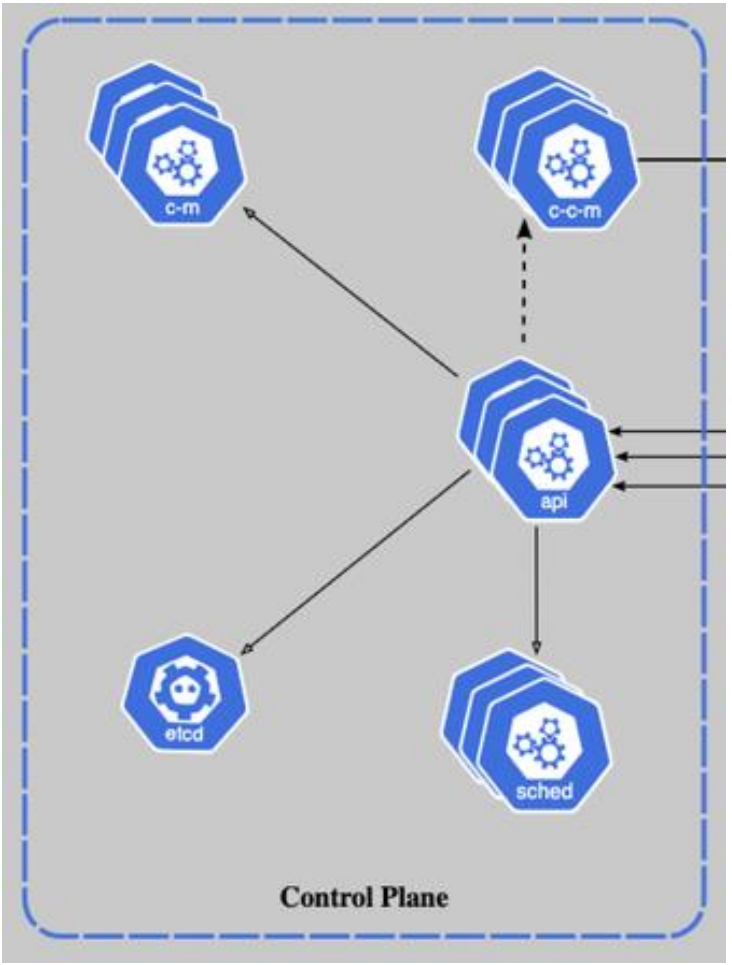
- API Server：整个集群的中心
  - 用户和k8s组件都只和API Server交互
  - Restful接口，提供了对集群内部对象的CRUD操作
  - 监听机制：当特定的对象状态发生变化时，通知controller manager
- Controller Manager
  - 管理一组Controllers，管理集群期望状态
  - 每一类K8s对象都会有相应的Controller进行管理，例如：Node Controller管理Node对象
  - 监听对象的状态，针对当前状态和期望状态进行操作



2.3 Kubernetes架构图

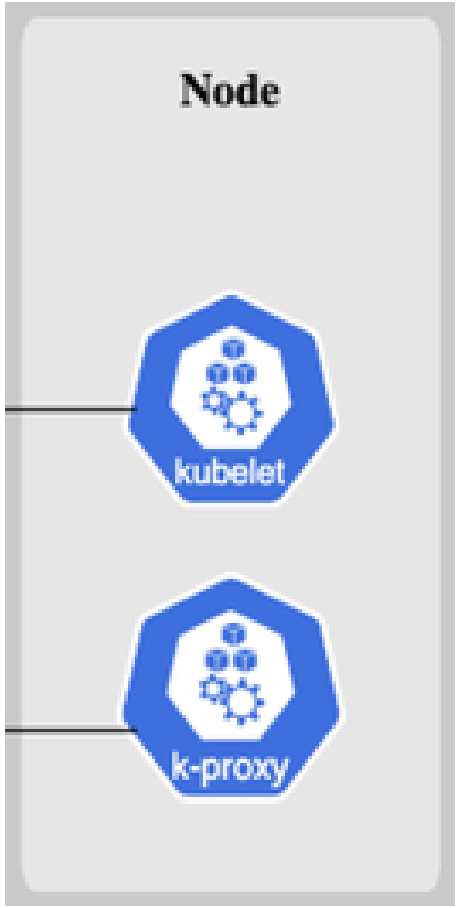
Control Plane

- etcd:存储集群数据的数据库
  - 存储整个k8s集群中所有有持久化需求的信息，保证集群崩溃后重启时能够恢复状态
  - 对象的Spec以及Status
  - 键值对存储（key-value store)
  - 强一致性，高可靠性
- scheduler:调度Pod
  - 通过API Server的监听机制，监听Pod的创建
  - 考虑具体任务的硬件需求、标签等因素，为Pod分配最合适的Node
  - 向API Server更新Pod的信息

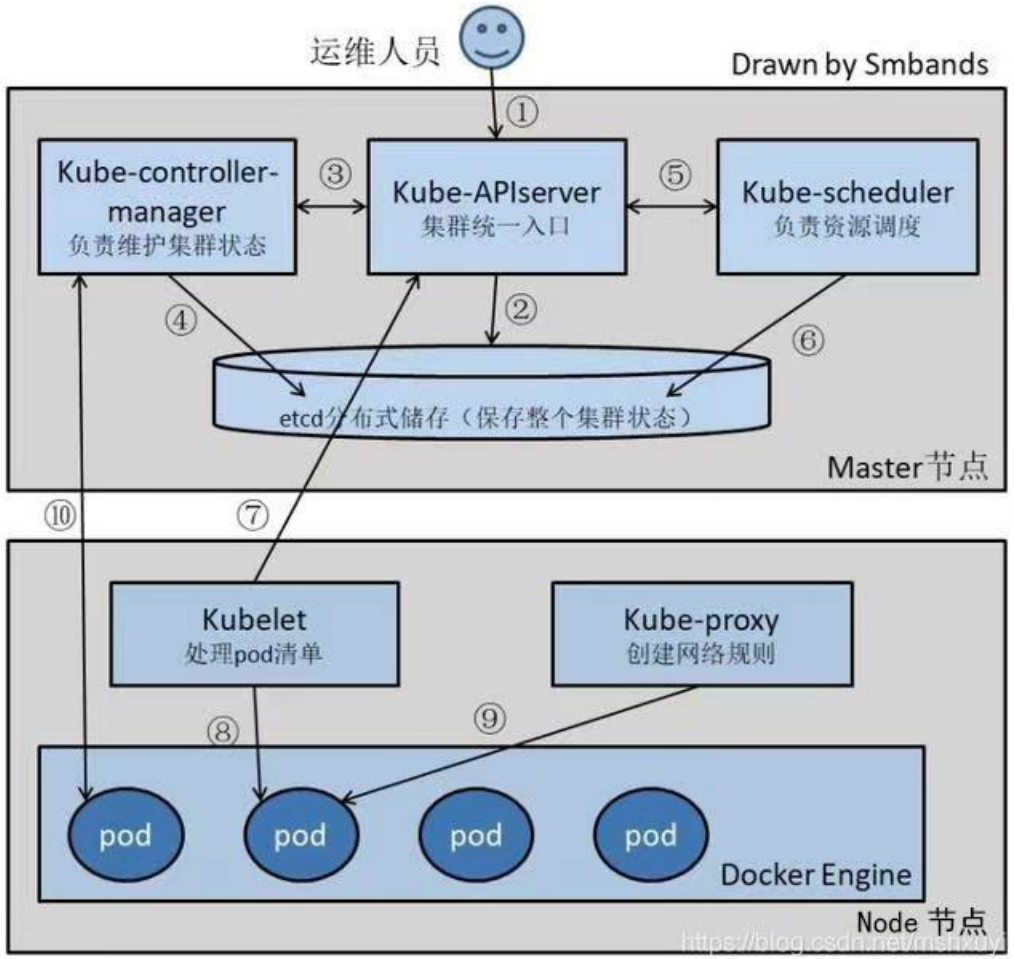


2.3 Kubernetes架构图

- Node（工作节点）
  - Kubelet
    - Node的实际管理者，负责真正地对Pod进行管理
    - 收集Node，Pod的状态，与API Server通信
  - kube-proxy
    - 网络代理
    - 确保Node、Pods间的网络通信



2.4 Kubernetes工作流程



1 用户对k8s执行操作的指令发送给api server

2 将用户指令的信息保存到etcd

3和4 与Controller manager通信，触发controller对集群的操作，通过api server写入信息

5和6 api server会将对应的pod需求告诉scheduler，进行资源调度，决定执行pod的node，并写入信息

7和8 kubelet收到任务要求之后，就会实际的创建pod，对pod进行管理

9 kube proxy建立与pod相关的网络规则

10 将pod和node的状态实时更新，让controller manager决定是否还需要进行下一步的操作

注：集群内其他组件都是只和api server通信的，图中这样画只是为了逻辑清晰

2.5 Kubernetes实践案例

- 本次案例，我们加入前端，并考虑后端应用的高可用性，使用K8s完善此前Python后端的部署方案。具体流程如下：
  - 前后端Docker镜像准备
  - K8s配置文件准备
  - 部署运行

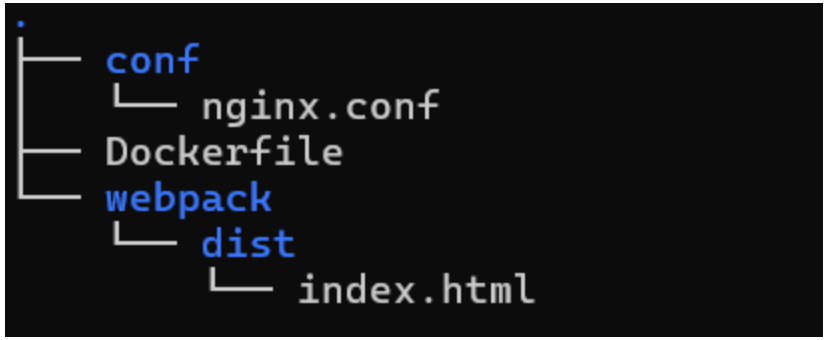
（注：本次实践略过K8s集群环境部署搭建过程，仅关注应用部署过程）

2.5 Kubernetes实践案例

前后端Docker镜像准备

后端镜像和此前一样，不再赘述。前端镜像构建过程与后端完全一致，该部分仅给出前端代码和Dockerfile的编写。

- 前端代码
  - 项目结构



1. conf文件夹存放nginx配置文件
2. webpack文件夹存放前端项目代码，静态页面等。
3. Dockerfile文件用于构建前端镜像。

2.5 Kubernetes实践案例

前后端Docker镜像准备

前端代码

- index.html

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>test</title>
    <script src="https://ajax.aspnetcdn.com/ajax/jquery/jquery-3.5.1.min.js"></script>
    <script>
        $(function () {
            test()
        })

        /*使用ajax向后端请求 计算1+1的结果*/
        function test() {
            $.ajax({
                url: "/api/get_api?num1=" + 1 + "&num2=" + 1,
                type: "GET",
                dataType: "json",
                success: function (result) {
                    $("#result").text("result: " + result["result"]) // 将结果放入下方p标签
                }
            })
        }
    </script>
</head>

<body>
    <h2>测试页面</h2>
    访问后台数据结果: <p id="result"></p>
</body>

</html>
```

- nginx.conf（仅列出核心反向代理部分）

```
server {
    listen       8090;
    server_name  localhost;
    location / {
        root   /usr/share/nginx/webpack/dist;
        index  index.html index.htm;
    }
    # 反向代理后端api
    location /api/ {
        #rewrite ^/api/(.*)$ /$1 break;
        proxy_pass http://<Node IP>:8080; # 此处替换为任意集群Node节点的IP
    }
}
```

- Dockerfile

```
# 以nginx:1.20.0-alpine为基础镜像
FROM nginx:1.20.0-alpine
# 覆盖nginx配置文件
COPY ./conf/nginx.conf /etc/nginx/nginx.conf
# 传入静态页面文件
COPY ./webpack/dist /usr/share/nginx/webpack/dist
```

2.5 Kubernetes实践案例

K8s配置文件准备

- 后端deployment.yaml文件

```yaml
# api-version 业务场景可选v1
apiVersion: v1
# 定义资源类型，deployment是副本控制器
kind: Deployment
# 定义资源的元数据
metadata:
  name: demo-backend # 定义资源的名称 同一namespace空间下必须唯一
  namespace: <namespace> # 指定命名空间 这里替换为自己的命名空间
  labels: # 定义Pod的标签
    app: demo-backend
# 定义该资源期望的参数属性
spec:
  # 定义标签选择器
  selector:
    app: demo-backend # 匹配上面定义的标签 需要与上面标签定义的app一致
  replicas: 2 # 定义副本数量 后端应用有两个副本进行负载均衡
  template: # 副本属性
    metadata:
      labels:
        app: demo-backend
    spec:
      # 定义容器属性
      containers:
      - name: demo-backend # 定义一个容器名 一个- name定义一个容器
        image: backendapp:v1 # 定义容器使用的镜像
        ports:
        - containerPort: 5000 # 定义容器对外端口
```

- 后端service.yaml文件

```yaml
apiVersion: v1
kind: Service
metadata:
  name: demo-backend-service
  namespace: <namespace>
spec:
  type: NodePort # 类型是NodePort （如果这里只是内部服务 可以使用ClusterIP）
  ports:
  # 目标：任何一个节点的IP:8090的请求都会映射到Pod:5000端口
  - port: 5000 # 该Service监听的端口
    targetPort: 5000 # 对应Pod（deployment）的端口
    name: backend-port
    nodePort: 8090 # Node（主机）的监听端口
  selector: # 标签选择器
    app: demo-backend # 后端deployment的label
```

2.5 Kubernetes实践案例

K8s配置文件准备

- 前端deployment.yaml文件

```yaml
apiVersion: v1
kind: Deployment
metadata:
  name: demo-frontend
  namespace: <namespace> # 替换为自己的命名空间（和后端一致）
spec:
  selector:
    matchLabels:
      app: demo-frontend
  replicas: 1 # 这里副本数设为1
  template:
    metadata:
      labels:
        app: demo-frontend
    spec:
      containers:
      - name: demo-frontend
        image: fronted:v1 # 指定前端镜像
        ports:
        - containerPort: 8080 # 前端容器暴露端口 （与nginx配置文件一致）
```

- 前端service.yaml文件

```yaml
apiVersion: v1
kind: Service
metadata:
  name: demo-frontend-service
  namespace: <namespace> # 替换为自己的namespace
spec:
  type: NodePort
  ports:
  # 目标 任意Node IP:8080 请求转发到 Pod的8080端口
  - port: 8080
    name: frontend-port
    targetPort: 8080
    nodePort: 8080
  selector:
    app: demo-frontend # 与fronted deployment中标签一致
```

2.5 Kubernetes实践案例

部署运行

- 使用kubectl apply命令依次应用以下yaml文件创建对应对象。
  - 后端service.yaml
  - 前端service.yaml
  - 后端deployment.yaml
  - 前端deployment.yaml

后续

- 访问：可以通过节点IP:8080访问前端应用

2.6 其他容器编排工具

- Docker Compose

  用于定义和运行多个Docker容器的工具。它允许用户通过单个YAML文件来定义多个容器的配置，并通过简单的命令来启动、停止和重启这些容器。相较于K8s局限性：

  - 单主机运行多个容器，适用于小型应用程序开发。
  - 扩展性和弹性有限，无法自动水平扩展或故障修复。

- Docker Swam

  Docker的原生容器集群管理工具，它将多个Docker主机组成一个集群，以实现Docker容器的高可用性和负载均衡。与K8s差异：

  - swarm与docker深度绑定，偏重容器的部署；k8s独立于docker，不完全依赖底层的docker，侧重业务应用的集群部署，docker仅是底层的一种技术支撑。
  - 场景规模与复杂度不同，swarm更适合中小规模、部署简单的场景；k8s更适合大规模复杂场景。

目录

- 1 微服务容器部署（Docker）
- 2 微服务容器编排（Kubernetes）
- 3 微服务系统运维（DevOps）
- 4 小结

3.1 DevOps

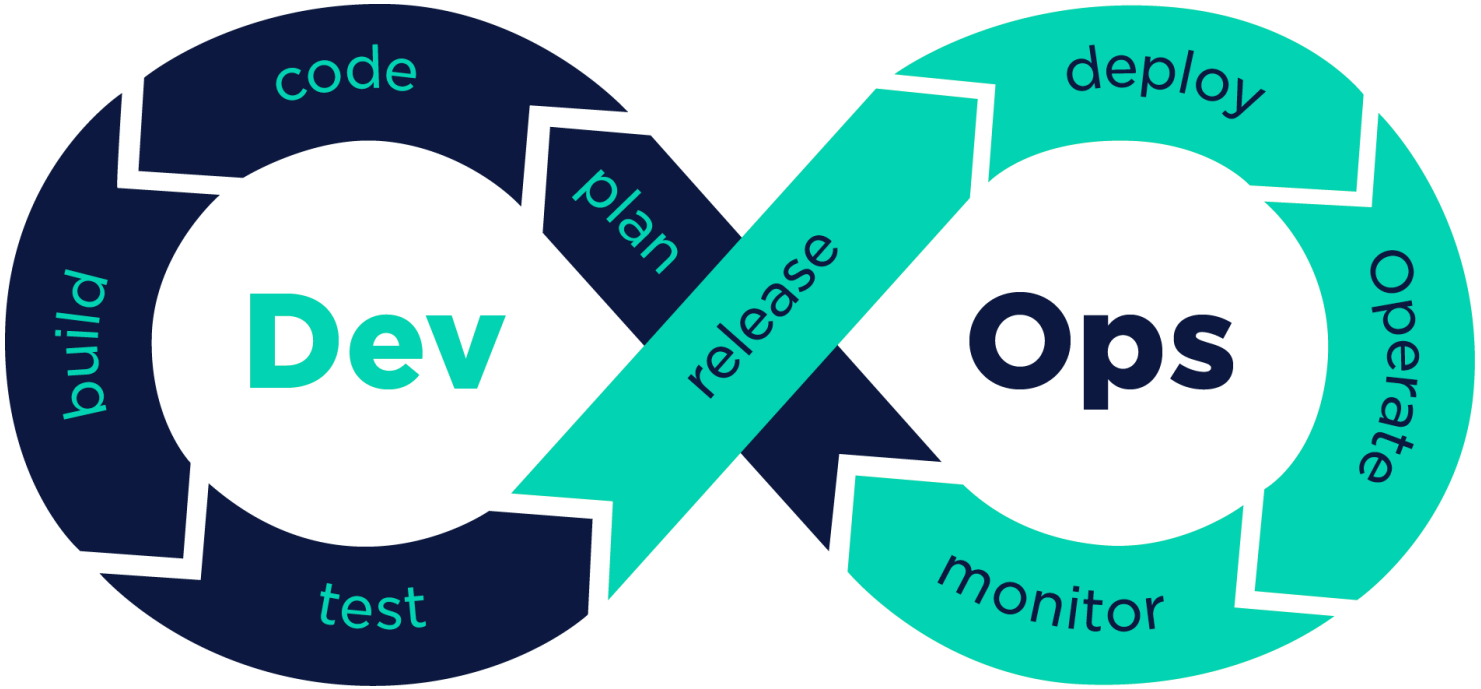
- DevOps = Development + Operations

- DevOps是一组过程、方法与系统的统称，用于促进开发、技术运营和质量保障部门之间的沟通、协作与整合。

- DevOps 源自于敏捷方法。 它添加了新的流程和工具，将 CI/CD 的持续迭代和自动化扩展到软件交付生命周期的其余部分。 它在流程中的每一步都实现了开发和运营之间的密切协作。

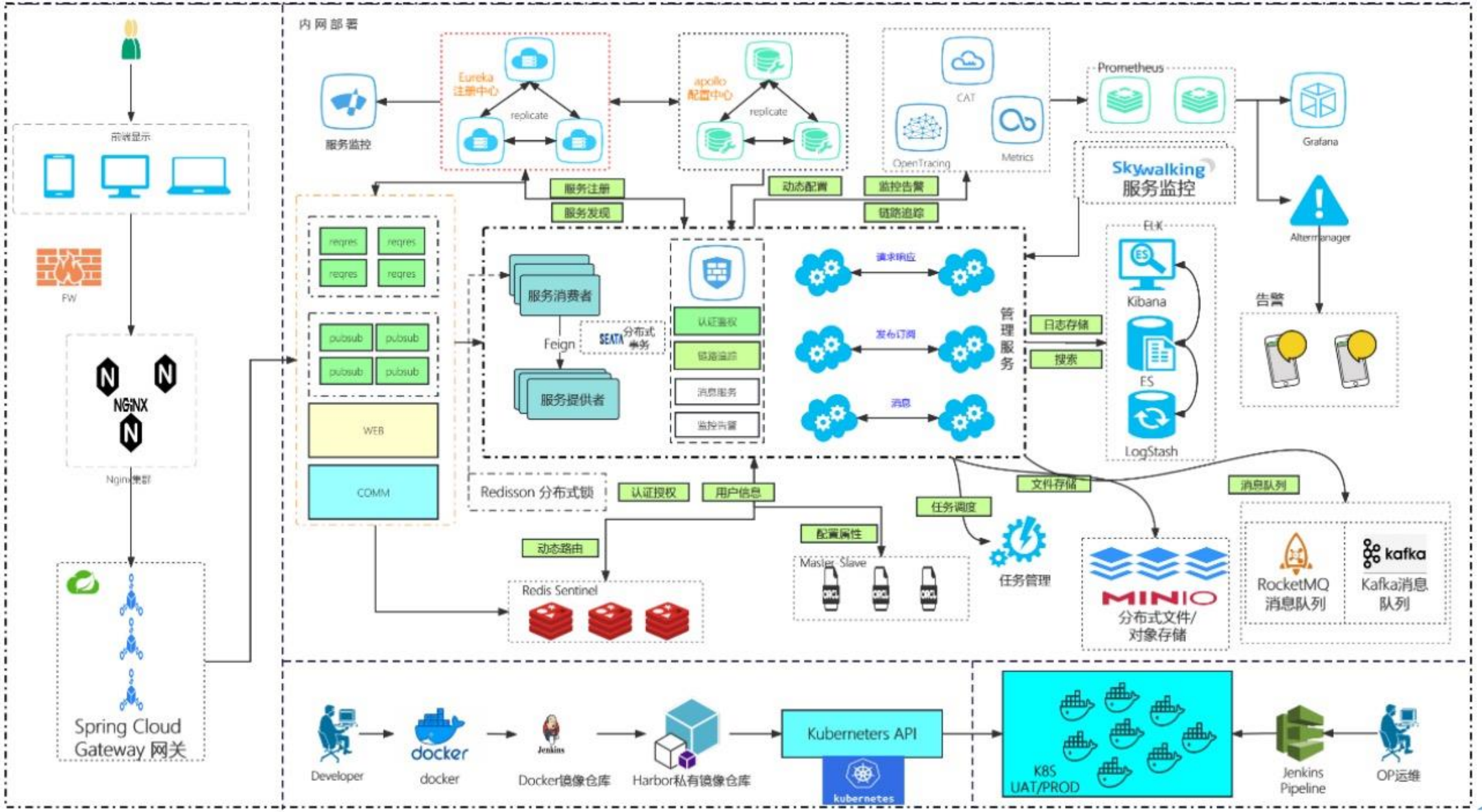
DevOps生命周期和工具链

- 规划
- 开发
- 集成(CI/CD)
- 部署(持续部署)
- 运营
- 学习（持续反馈）



- 规划工具
- 源代码控制
- 构建工具
- 测试工具
- 持续集成和部署（CI/CD）
- 配置管理工具
- 云平台
- 容器调度程序
- 监控和记录工具

3.2 微服务的全生命周期管理



3.3 规划工具

项目计划、任务分配、需求管理、错误跟踪

Jira Software

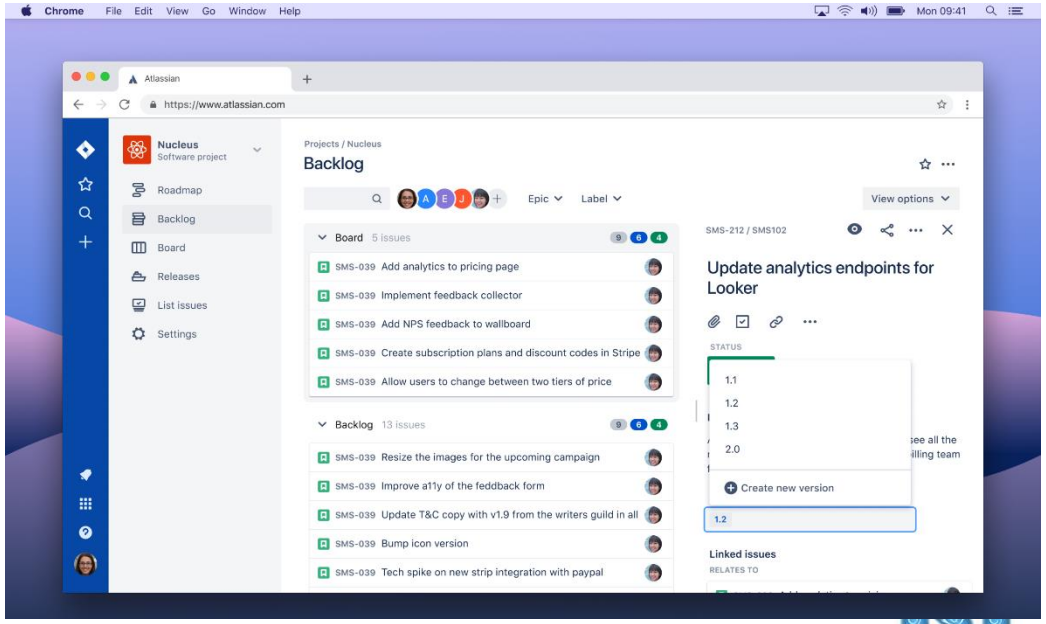
GitLab

特点

- 创建、分配、跟踪和管理任务。
- 识别、记录和跟踪软件缺陷。
- 支持 Scrum、Kanban 等敏捷方法。

功能

- 看板视图：实时监控项目进度
- 报告和分析：生成项目状态和性能报告
- 集成工具：与其他生命周期工具的第三方应用集成



3.4 源代码控制



- **代码仓库：**
- 提供 Git 仓库管理功能，支持代码版本控制。
- **CI/CD：**
- 自动化的持续集成和持续部署功能，简化开发流程。
- **代码审查：**
- 内置的代码审查工具，促进团队合作和代码质量。
- **项目管理：**
- 任务跟踪、看板、里程碑等项目管理工具。
- **集成扩展：**
- 支持与其他工具和服务的集成，如 Kubernetes、JIRA 等。



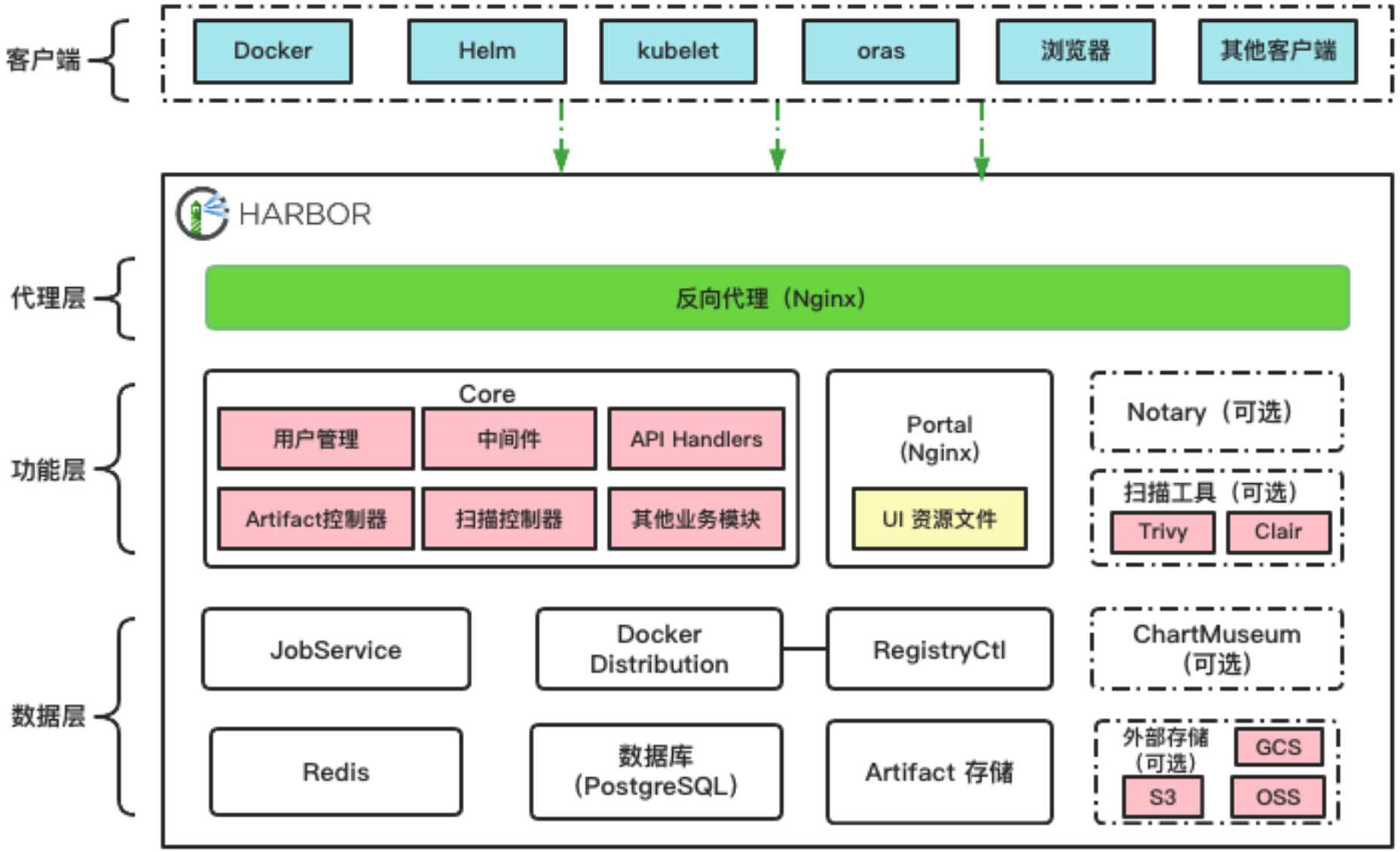




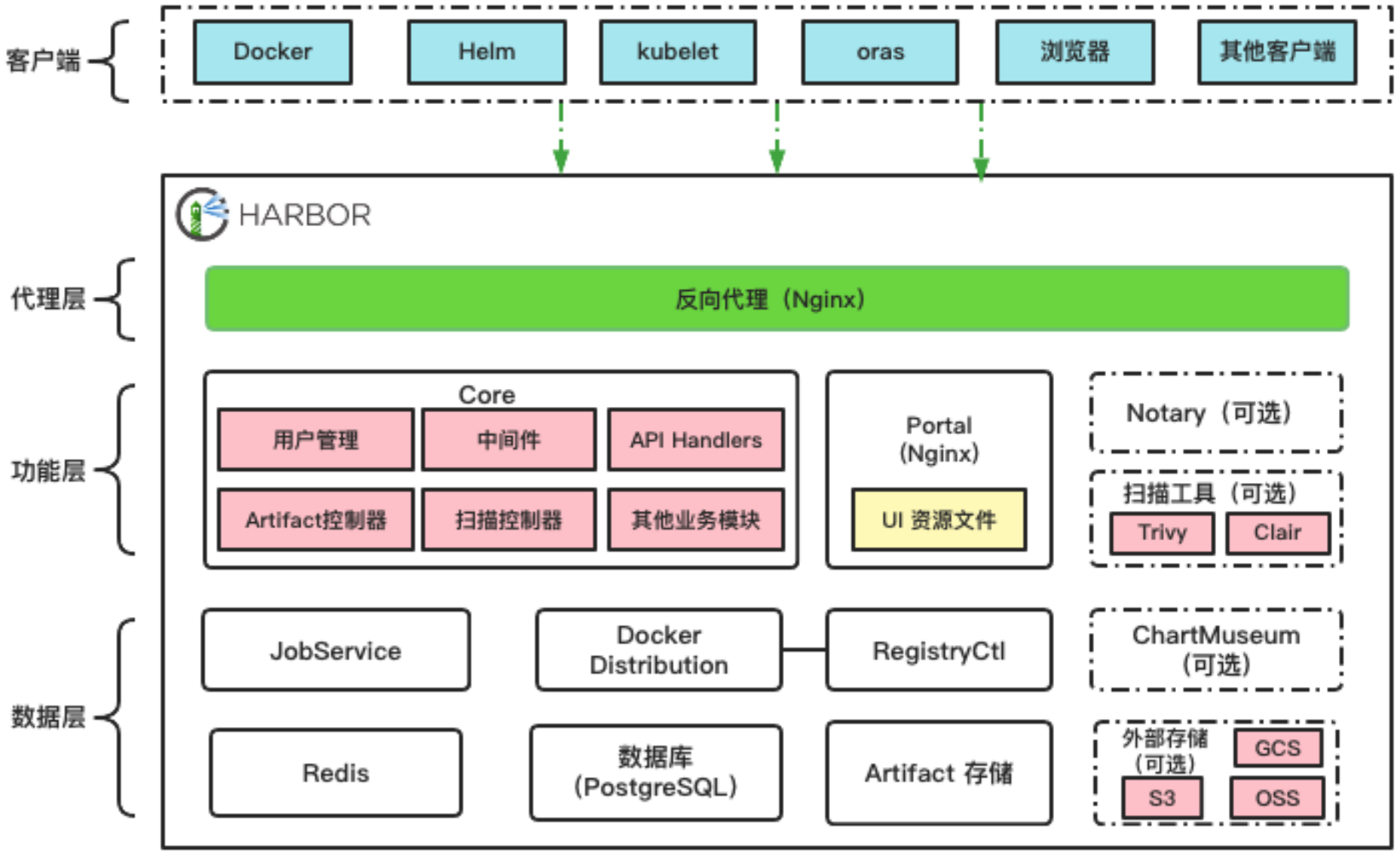
镜像仓库Harbor

- Harbor是一个由VMware用于存储和分发Docker镜像的企业级Registry服务器，其中的Docker镜像及其元数据被作为artifact管理。
- Harbor 由 5 个容器组成：
  - proxy：由 Nginx 服务器构成的反向代理。
  - registry：由 Docker 官方的开源 registry 镜像构成的容器实例。
  - ui：即架构中的 core services, 构成此容器的代码是 Harbor 项目的主体。
  - mysql：由官方 MySql 镜像构成的数据库容器。
  - log: 运行着 rsyslogd 的容器，通过 log-driver 的形式收集其他容器的日志

Harbor架构

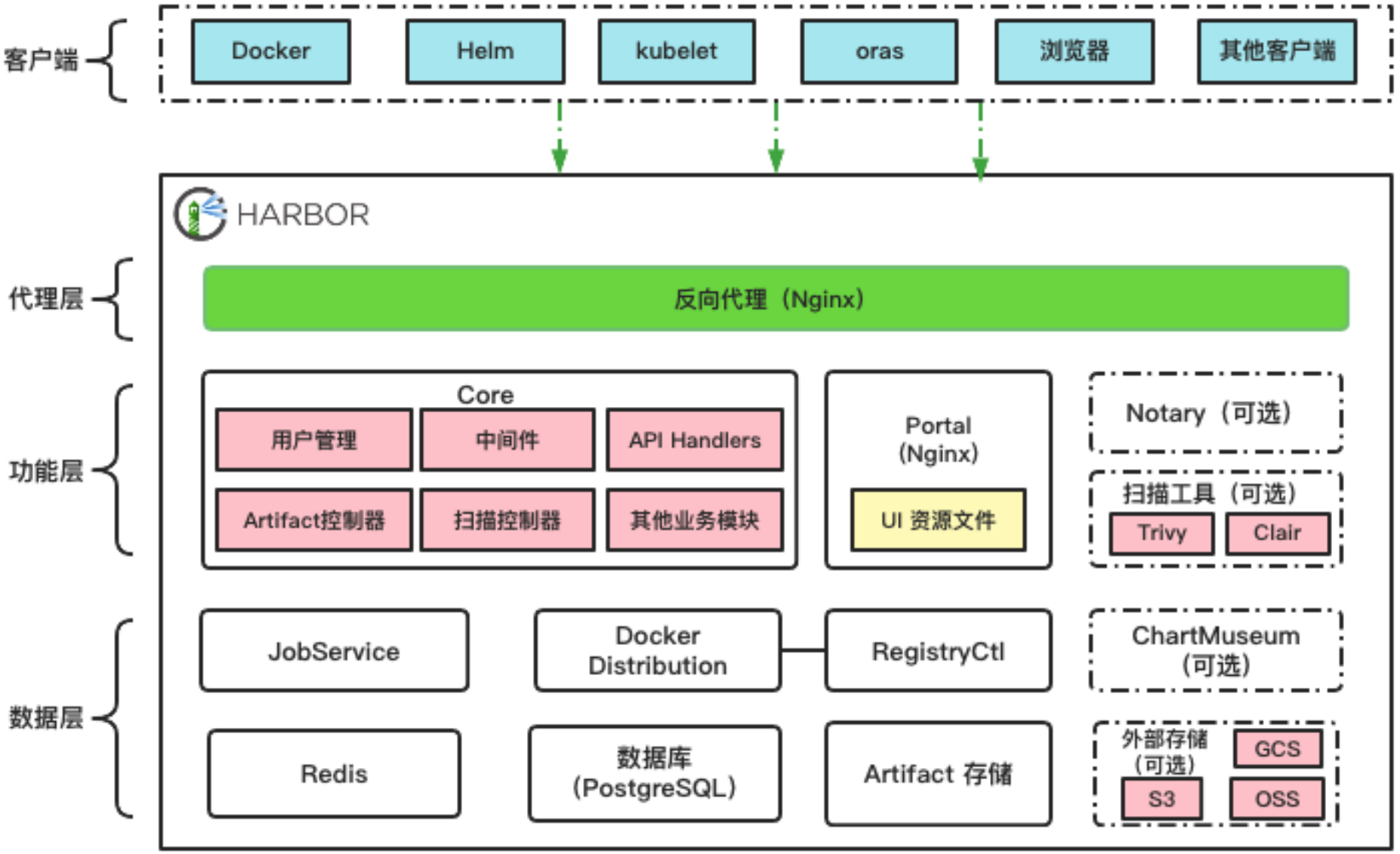

客户端
Docker
Helm
kubelet
oras
浏览器
其他客户端
HARBOR
代理层
反向代理（Nginx）
功能层
Core
用户管理
中间件
API Handlers
Artifact控制器
扫描控制器
其他业务模块
Portal
(Nginx)
UI 资源文件
Notary（可选）
扫描工具（可选）
Trivy
Clair
数据层
JobService
Docker
Distribution
RegistryCtl
ChartMuseum
(可选)
Redis
数据库
(PostgreSQL)
Artifact 存储
外部存储
(可选)
GCS
S3
OSS

Harbor架构



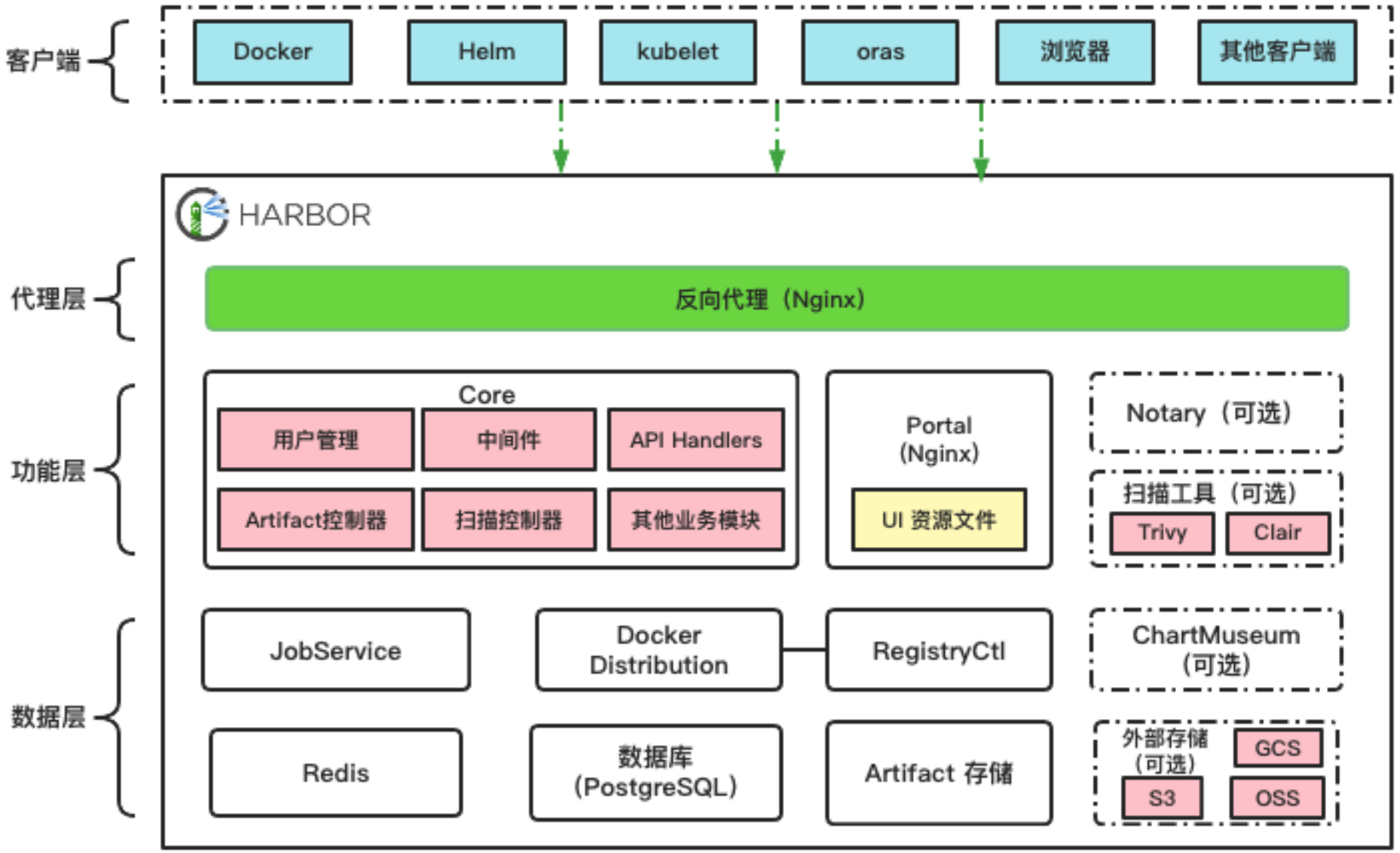
代理层：Nginx 反向代理，负责接收不同类型的客户端请求，包括浏览器、用户脚本、Docker 等，并根据请求类型和 URI 转发给不同的后端服务进行处理。

Harbor架构



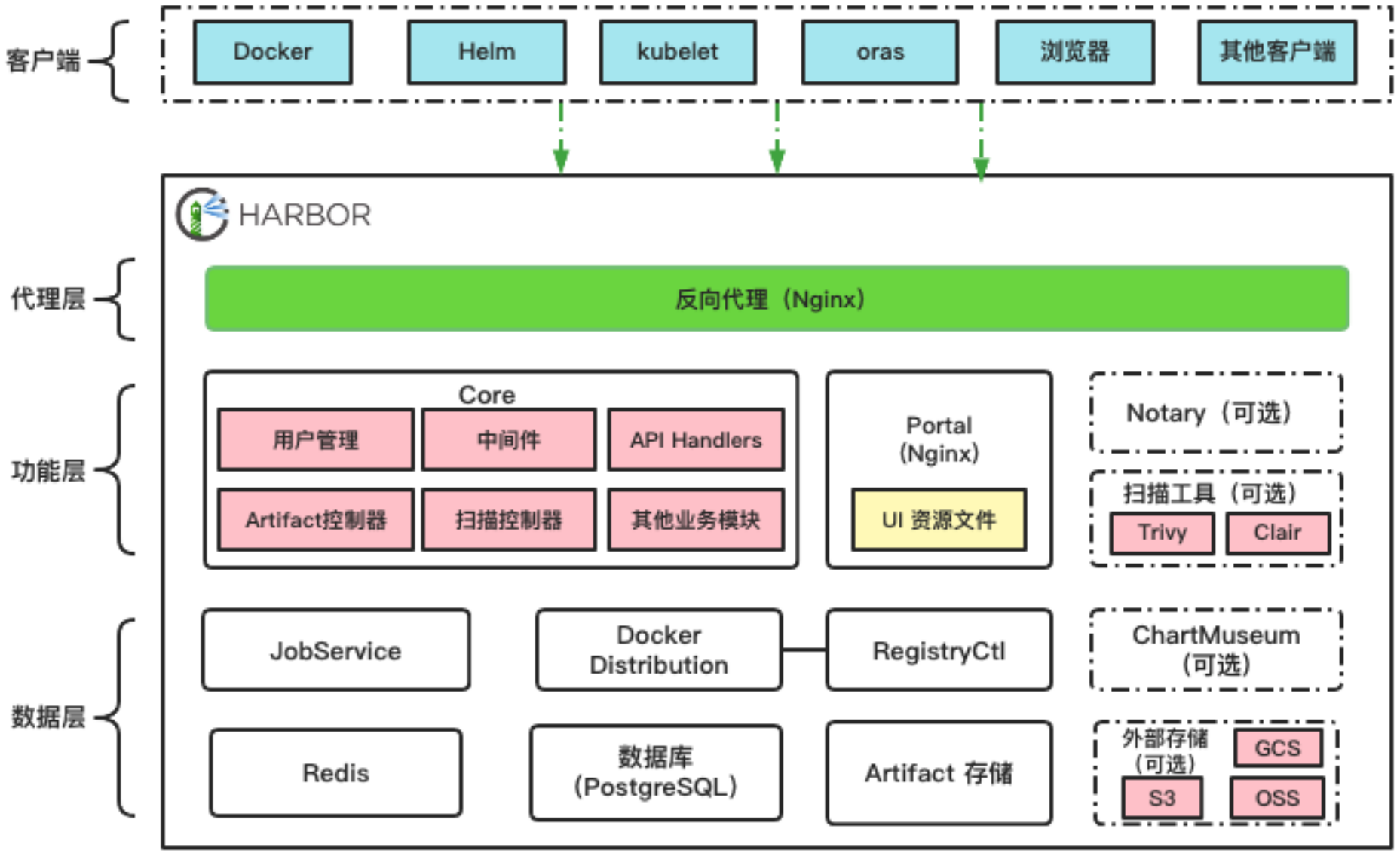
Portal：是一个基于 Argular 的前端应用，提供 Harbor 用户访问的界面。

Harbor架构



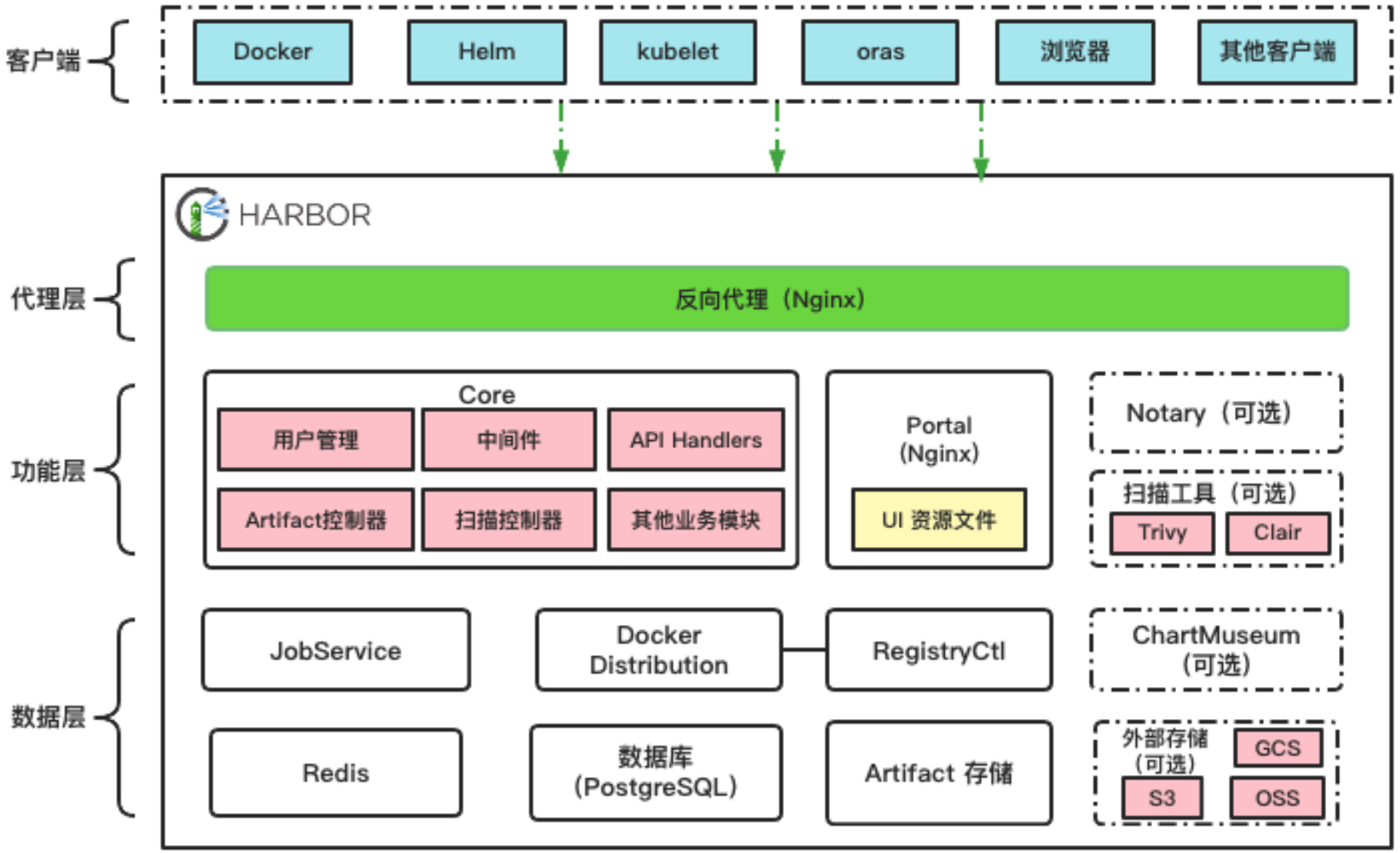
Core：是 Harbor 中的核心组件，封装了 Harbor 绝大部分的业务逻辑。

Harbor架构



JobService：异步任务组件，负责 Harbor 中很多比较耗时的功能，比如 Artifact 复制、扫描、垃圾回收等。

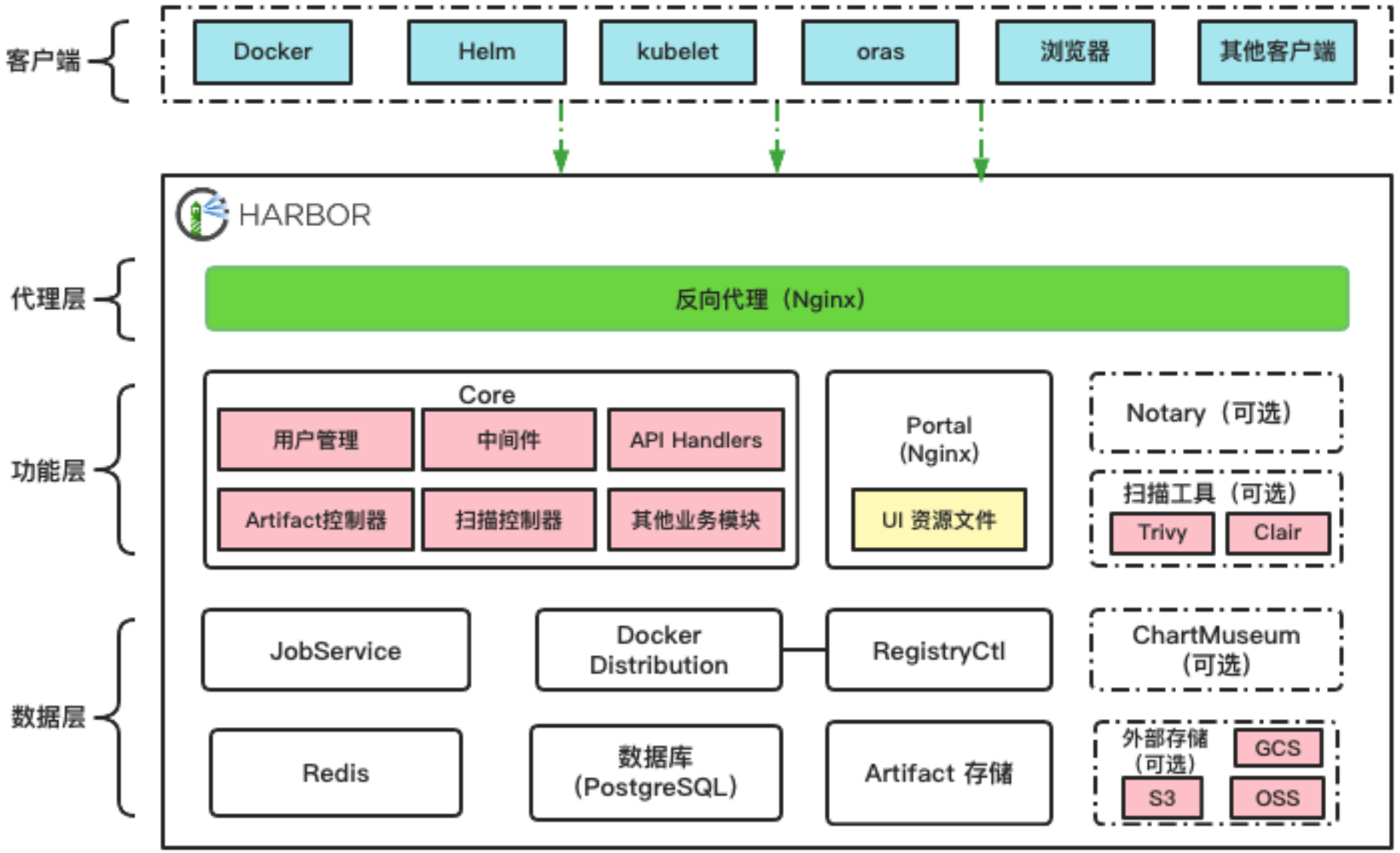
Harbor架构



Docker Distribution：Harbor 通过 Distribution 实现 Artifact 的读写和存取等功能。

RegistryCtl： Docker Distribution 的控制组件。

Harbor架构

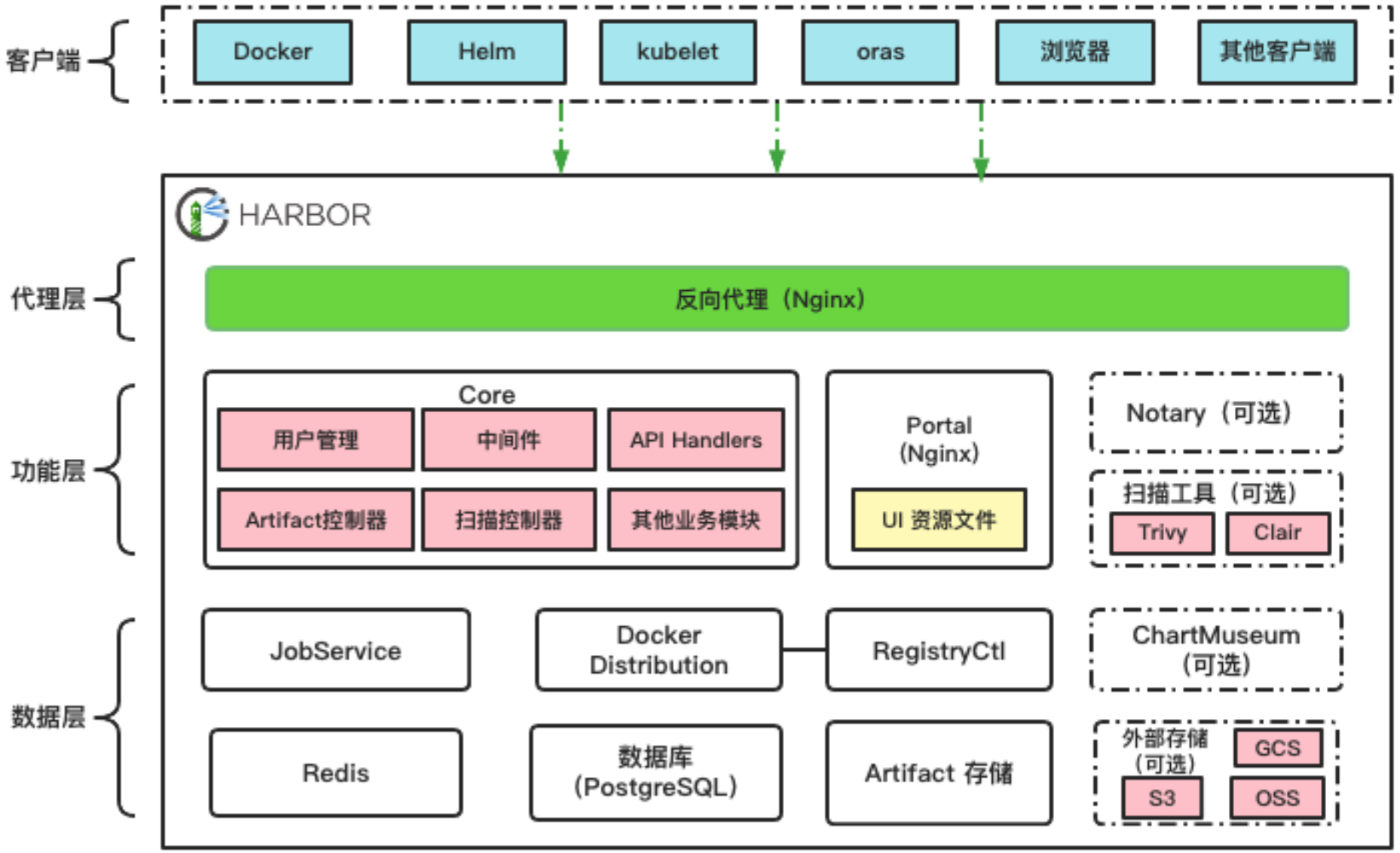


Notary：基于 TUF 提供镜像签名管理的功能。

扫描工具：镜像的漏洞检测工具。

ChartMuseum：提供 API 管理非 OCI 规范的 Helm Chart，随着兼容 OCI 规范的 Helm Chart 在社区上被更广泛地接受，Helm Chart 能以 Artifact 的形式在 Harbor 中存储和管理，不再依赖 ChartMuseum，因此 Harbor 可能会在后续版本中移除对 ChartMuseum 的支持。

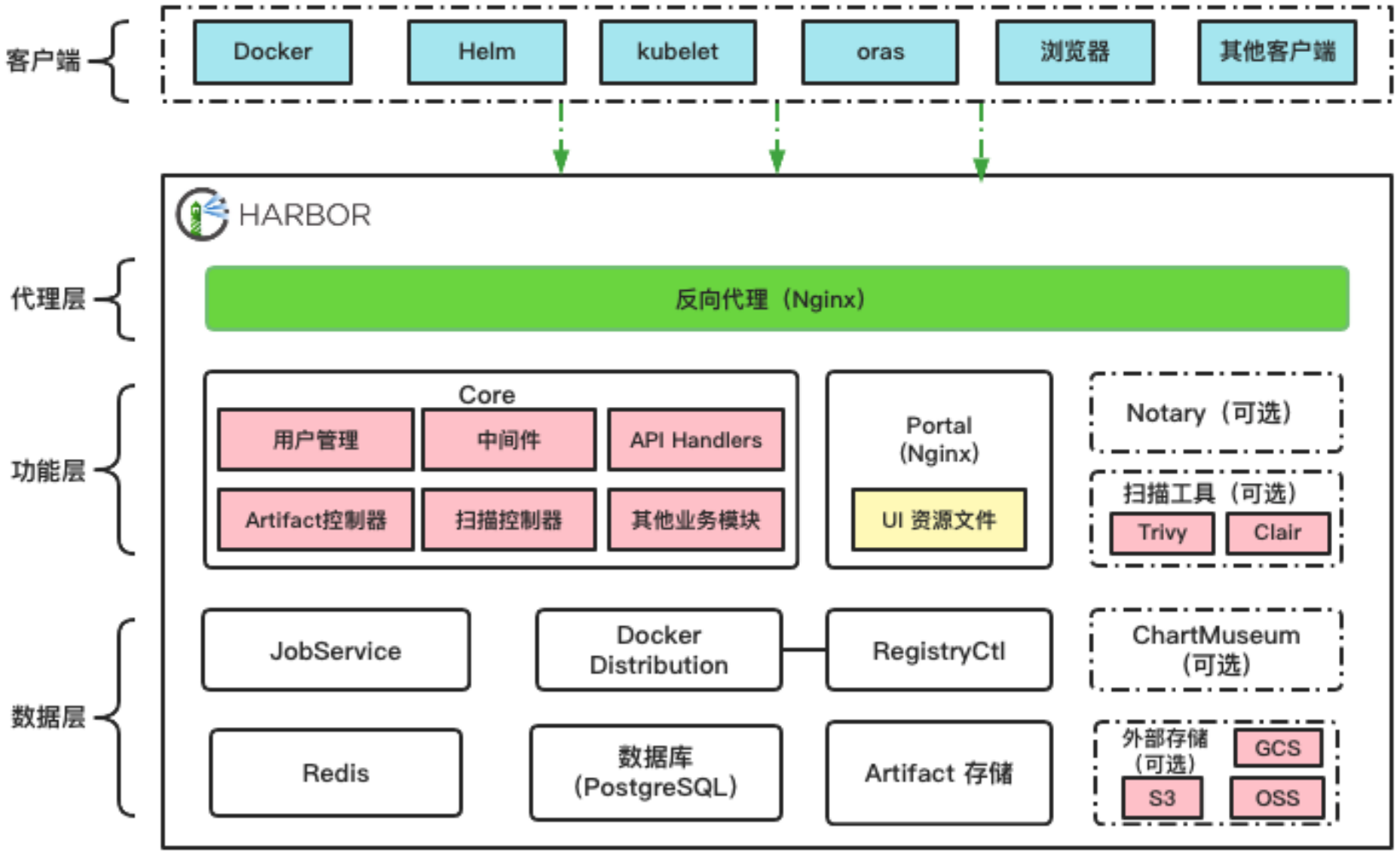
Harbor架构



Redis：主要作为缓存服务存储一些生命周期较短的数据，同时对于 JobService 还提供了类似队列的功能。

PostgreSQL：存储 Harbor 的应用数据，比如项目信息、用户与项目的关系、管理策略、配置信息、Artifact 的元数据等等。

Harbor架构



Artifact 存储：存储 Artifact 本身的内容，也就是每次推送镜像、Helm Chart 或其他 Artifact 时，数据最终存储的地方。默认情况下，Harbor 会把 Artifact 写入本地文件系统中。用户也可以修改配置，将 Artifact 存储在外部存储中，例如亚马逊的对象存储 S3、谷歌云存储 GCS、阿里云的对象存储 OSS 等等。

3.5 测试工具



- **注解驱动：**
  - 使用注解标记测试方法和生命周期。
- **断言方法：**
  - 提供一套丰富的断言方法，验证代码行为。
- **报告生成：**
  - 自动生成详细的测试报告，方便问题定位。
- **持续集成：**
  - 与 CI/CD 工具集成，实现自动化测试流程。

3.6 持续集成和部署（CI/CD）





➢ CI

持续集成（CI）是一种软件开发实践，将开发者的代码持续集成到共享代码库中，自动构建和测试，以确保每次代码更改都能正常工作。

➢ CD

持续交付（CD）扩展了CI的概念，旨在自动化软件交付流程，包括构建、测试、部署和监控，以确保新功能或修复能够迅速交付到生产环境。

持续集成

- 定义：自动化的代码合并过程，促进频繁的代码更改合并。
- 目的：确保代码更改的可靠性。

挑战

- 多人并行开发：多个开发人员同时修改同一应用的不同特性。
- 合并日问题：一次性合并所有代码可能导致冲突，工作繁琐。

优势

- 自动化测试：合并后自动进行各种测试，如单元和集成测试。
- 快速修复：自动化测试能快速发现并修复新旧代码之间的冲突。

CD

- "CD"在CI/CD中指的是持续交付和/或持续部署，这两个相关概念有时会被交替使用。两者都是关于自动化流水线的后续阶段，但有时会分开使用，以说明正在发生的自动化程度。选择持续交付还是持续部署取决于开发团队和运维团队的风险容忍度和具体需求。

持续交付

- 自动化从CI后的代码验证到仓库的发布过程。

特点

- 运维快速部署
- 自动化流程

目的

- 减少沟通障碍
- 提高部署效率

持续部署

- 自动化将代码从仓库发布到生产环境。

特点

- 实时上线
- 自动化测试
- 分段发布更改，降低风险

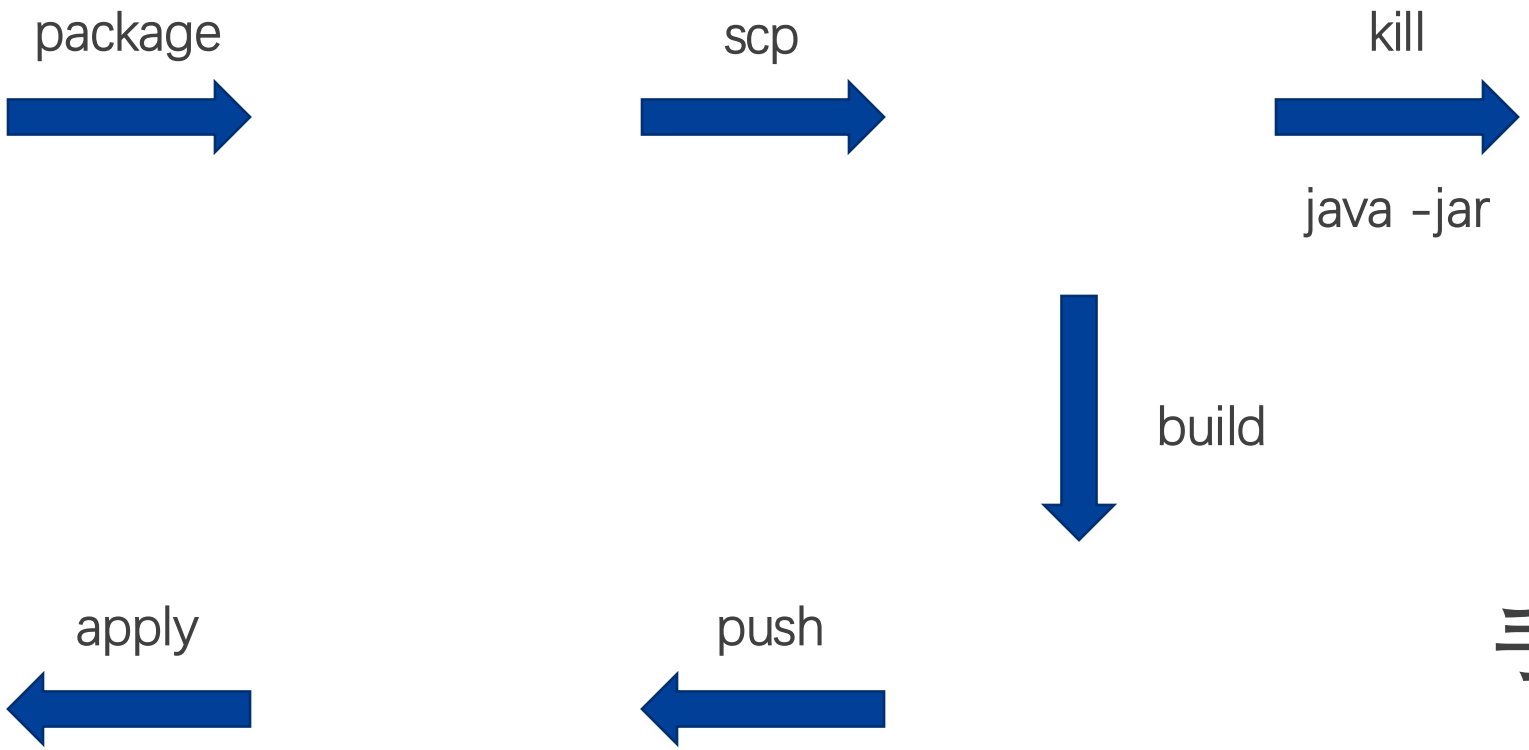
挑战

- 需要编写适应各种测试和发布阶段的自动化测试和自动部署脚本

CI/CD的工作原理


CONTINUOUS INTEGRATION
CONTINUOUS DELIVERY
CONTINUOUS DEPLOYMENT
BUILD
TEST
MERGE
AUTOMATICALLY RELEASE TO REPOSITORY
AUTOMATICALLY DEPLOY TO PRODUCTION

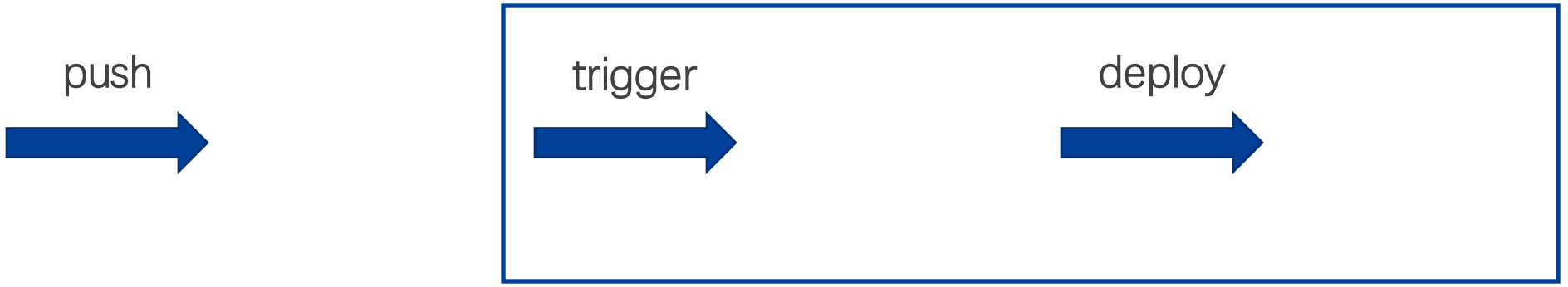
CI/CD的重要性



手动部署

枯燥

CI/CD的重要性

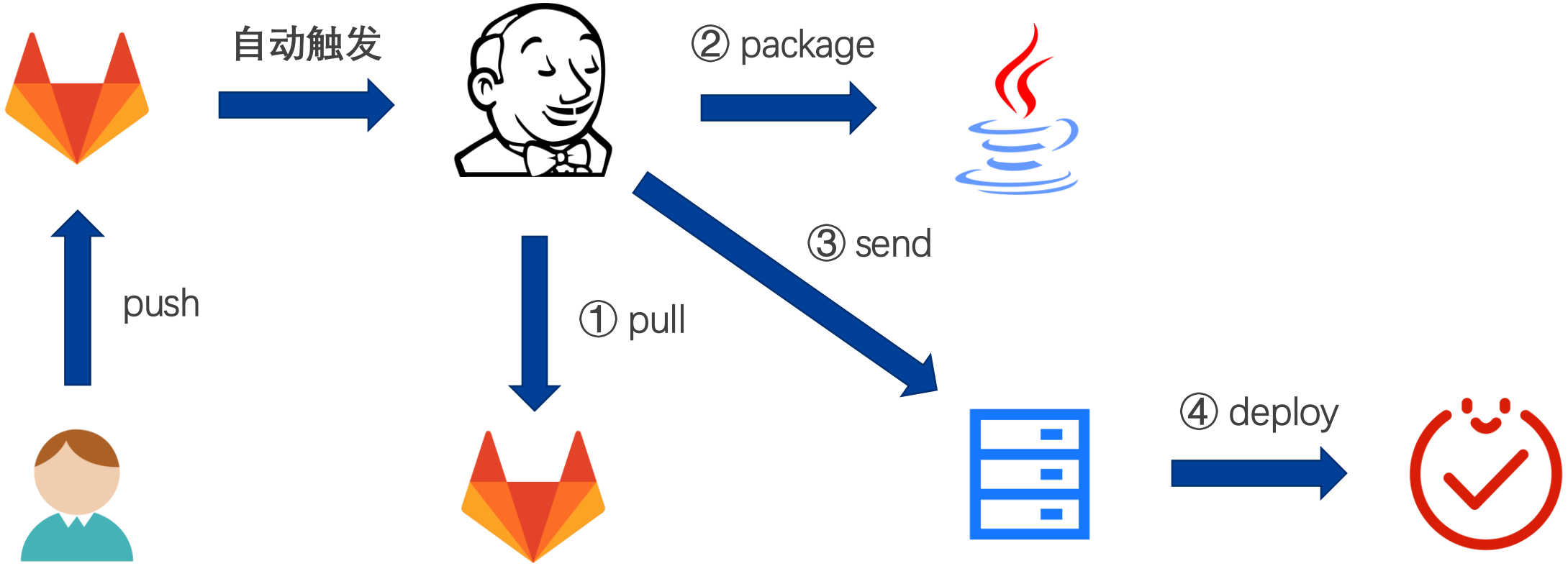


Push Is All You Need

团队收益：所有人可以追踪到项目的最新进度

个人收益：节省了部署所需的时间

CI/CD的重要性



CI/CD的重要性

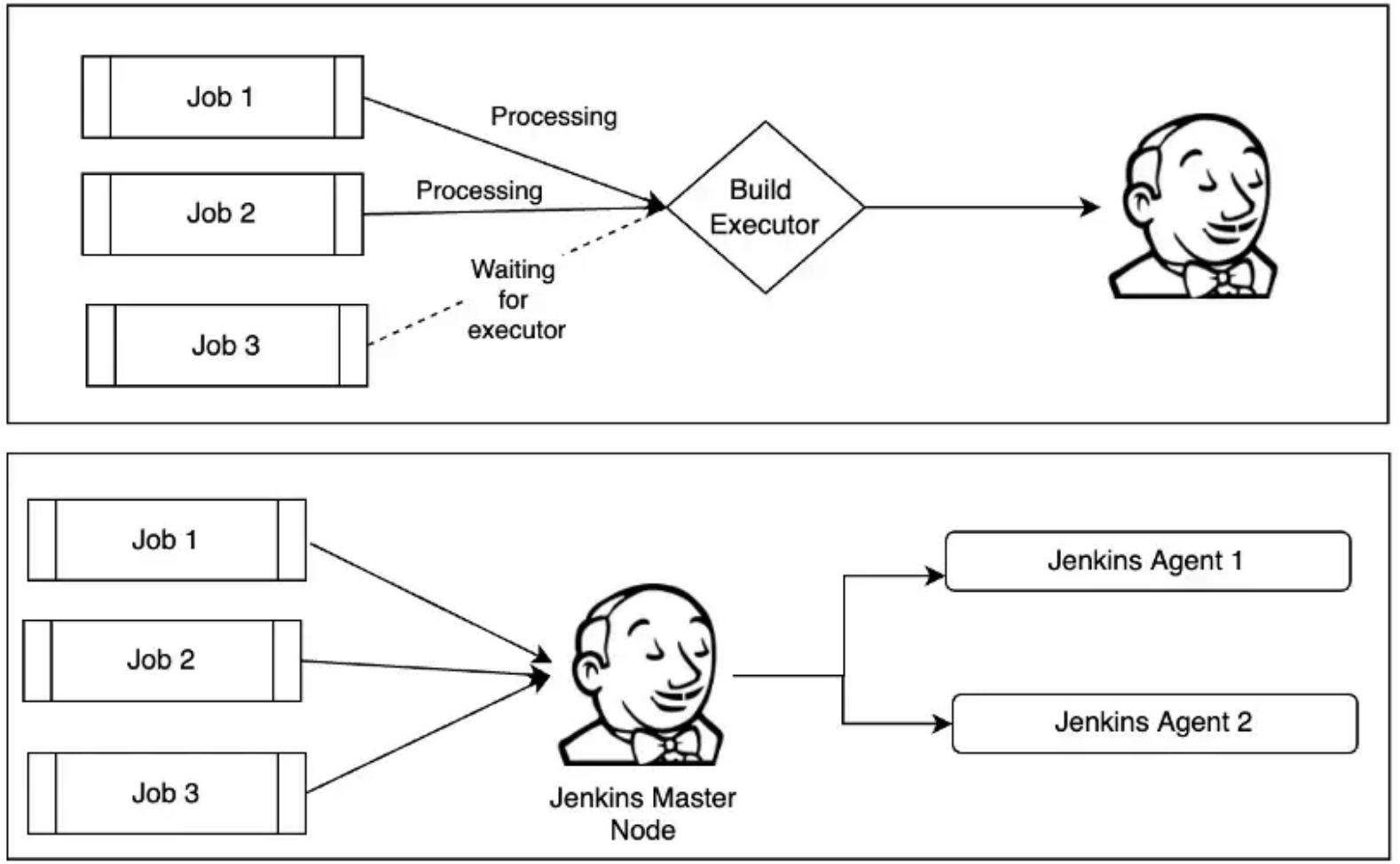
- **快速交付**
- CI/CD减少了手动干预，加速了代码的构建、测试和部署，缩短了交付周期。

- **错误减少**
- 自动化测试和部署减少了人为错误的风险，提高了软件质量。

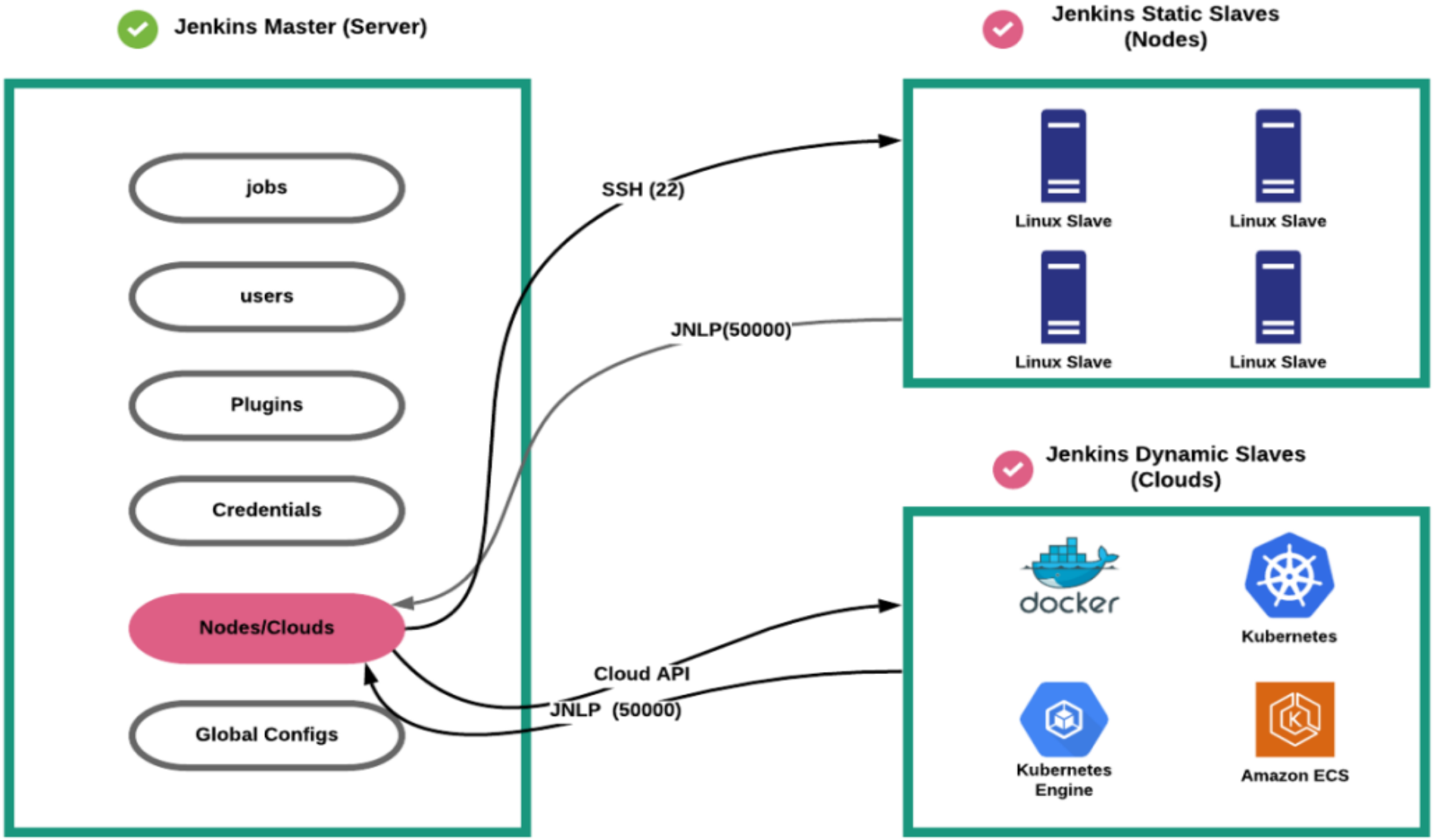
- **持续反馈**
- CI/CD流程提供了快速的反馈，使开发者能够快速识别和修复问题。

Jenkins架构

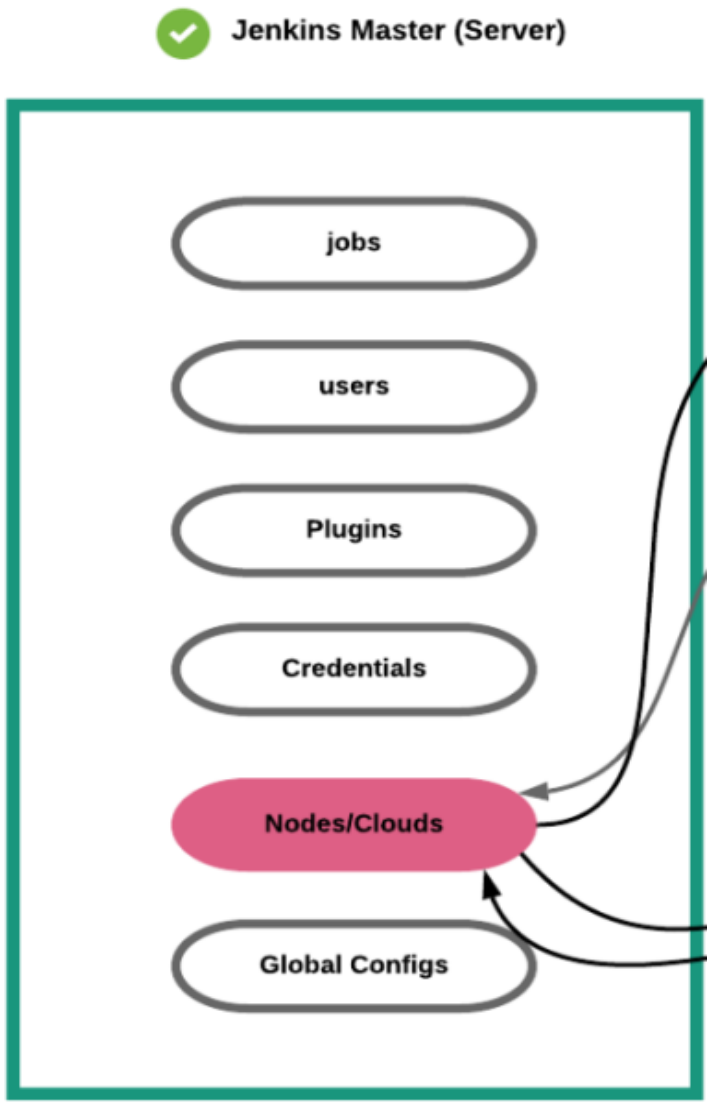
Jenkins通常以单节点模式工作，但其也可以通过代理的方式实现多节点架构，从而能够横向扩展Jenkins系统，支持大规模CICD流水线。



Jenkins架构


Jenkins Master (Server)
jobs
users
Plugins
Credentials
Nodes/Clouds
Global Configs
SSH (22)
JNLP(50000)
Cloud API
JNLP (50000)
Jenkins Static Slaves (Nodes)
Linux Slave
Linux Slave
Linux Slave
Linux Slave
Jenkins Dynamic Slaves (Clouds)
docker
Kubernetes
Kubernetes Engine
Amazon ECS

Jenkins Master



- Jobs是一系列步骤的集合，可用于构建源代码，测试代码，运行Shell脚本或在远程主机中运行Ansible角色。有多种工作类型可支持工作流程，以实现持续集成和持续交付。

- 插件是社区开发的模块，您可以将其安装在Jenkins服务器中。它使您可以添加Jenkins本身无法提供的更多功能。您还可以开发自定义插件。从Jenkins插件索引中查看所有插件 。

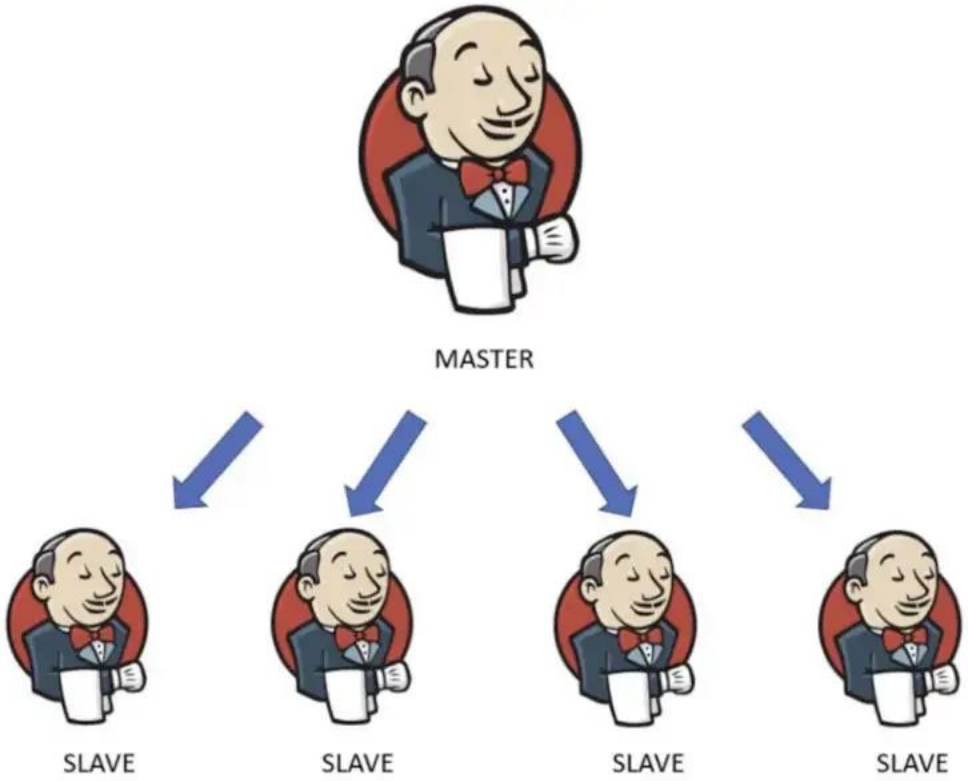
- 对要保存作业中的秘密信息进行加密，并存储为凭证。

- 在Jenkins主节点中，组件的所有配置都将作为配置文件。

提供有关所有Jenkins服务器操作的日志信息，包括作业日志，插件日志，webhook日志等。

Jenkins Slave

- Jenkins从节点是Jenkins服务器中的工作节点。
  - **静态从节点：**配置为静态从节点的服务器（Windows / Linux）。一直处于运行状态，并保持与Jenkins服务器的连接。
  - **动态从节点：** Jenkins Cloud从节点是具有动态的。每当触发作业时，从节点将按需部署为VM / 容器，并在作业完成后被删除。当拥有庞大的Jenkins生态系统并持续构建时，此方法可节省基础设施成本。



Jenkins优势

1. **持续集成/持续部署（CI/CD）**

   Jenkins支持持续集成和持续部署，可以自动触发构建、测试和部署任务，确保代码的质量和应用的可靠性。

2. **插件生态系统**

   Jenkins有一个丰富的插件生态系统，用户可以根据需要选择和安装各种插件来扩展其功能。

3. **易于配置**

   Jenkins提供了一个Web界面和一个Groovy-based DSL，使得配置自动化任务变得相对简单。

4. **多平台支持**

   Jenkins可以在多种操作系统上运行，包括Linux、Windows和macOS。

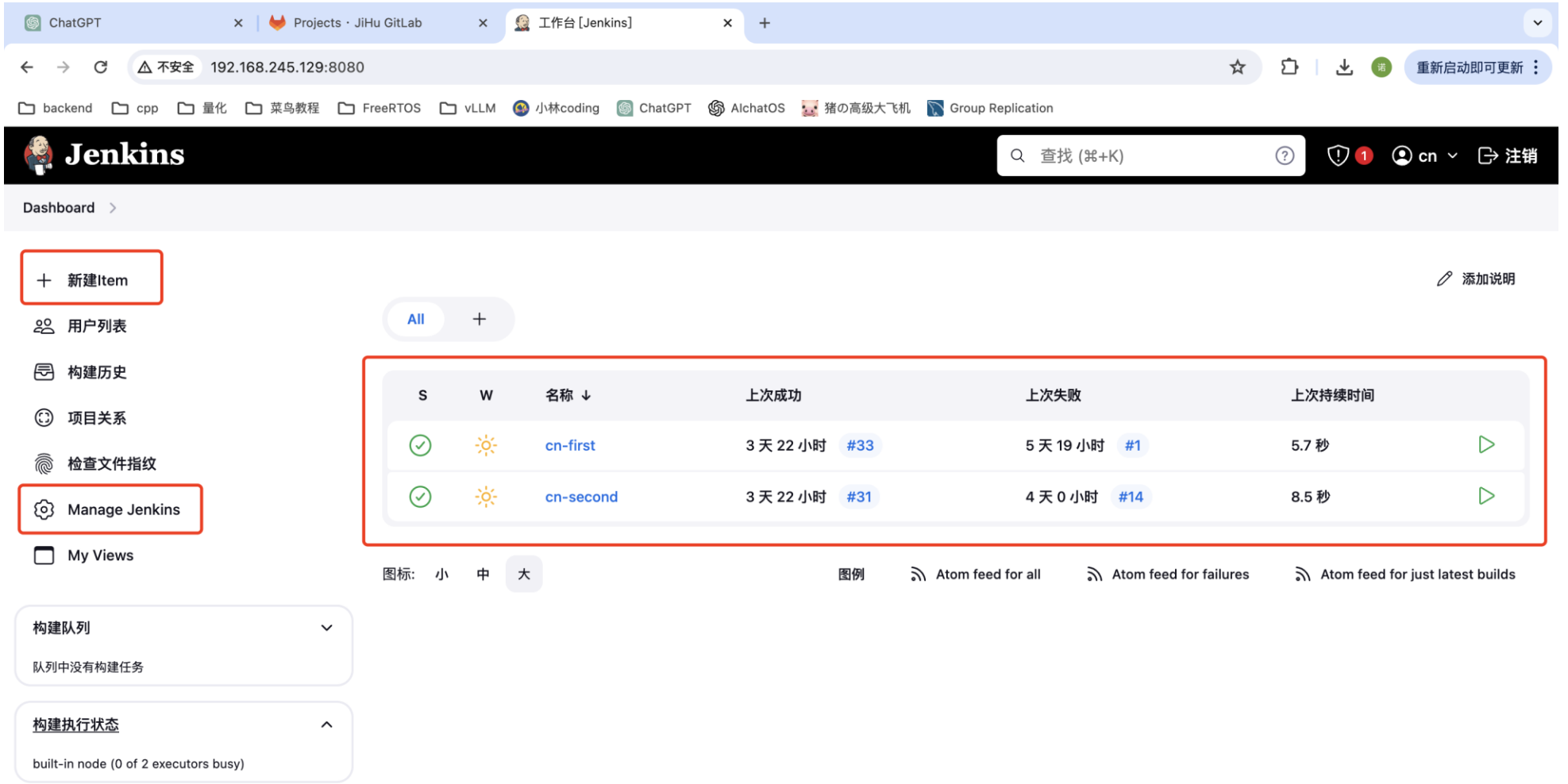
5. **可扩展性**

   除了内置的功能外，Jenkins还支持自定义脚本和扩展，满足不同项目和组织的特定需求。

6. **社区支持**

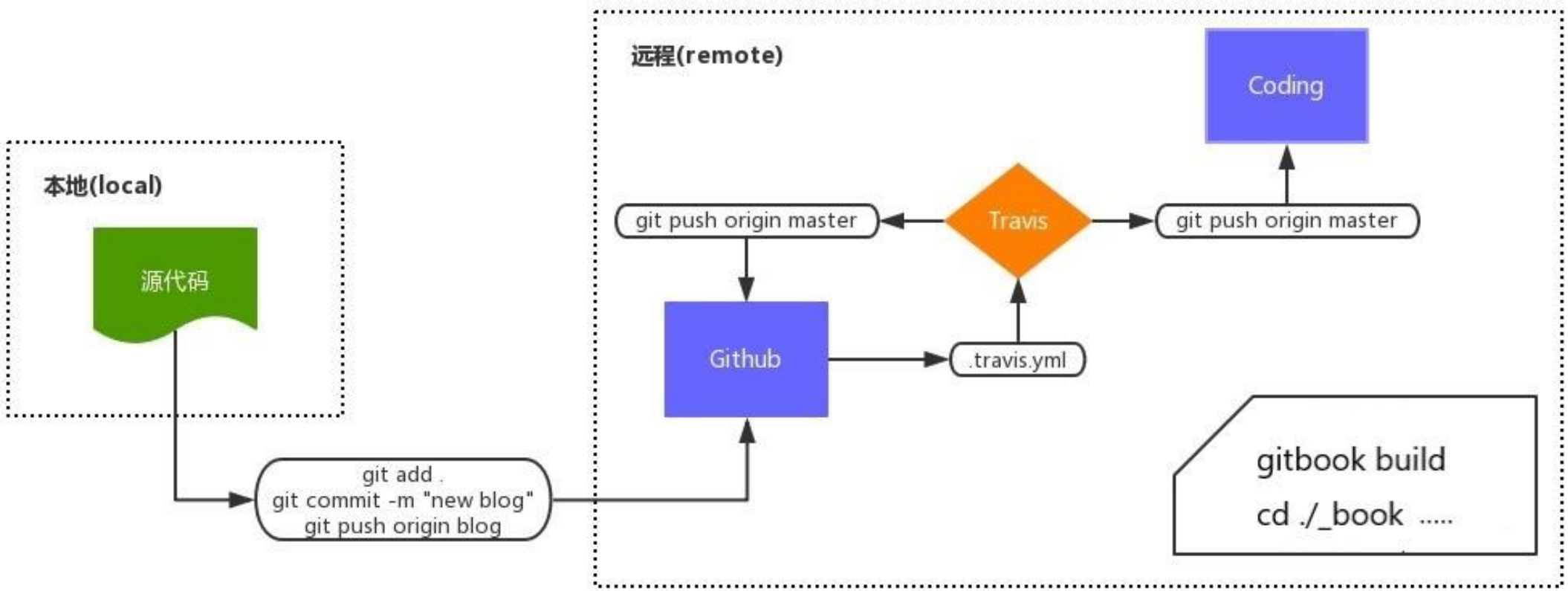
   作为一个开源项目，Jenkins拥有一个活跃的社区，提供了丰富的文档、教程和支持。

Jenkins

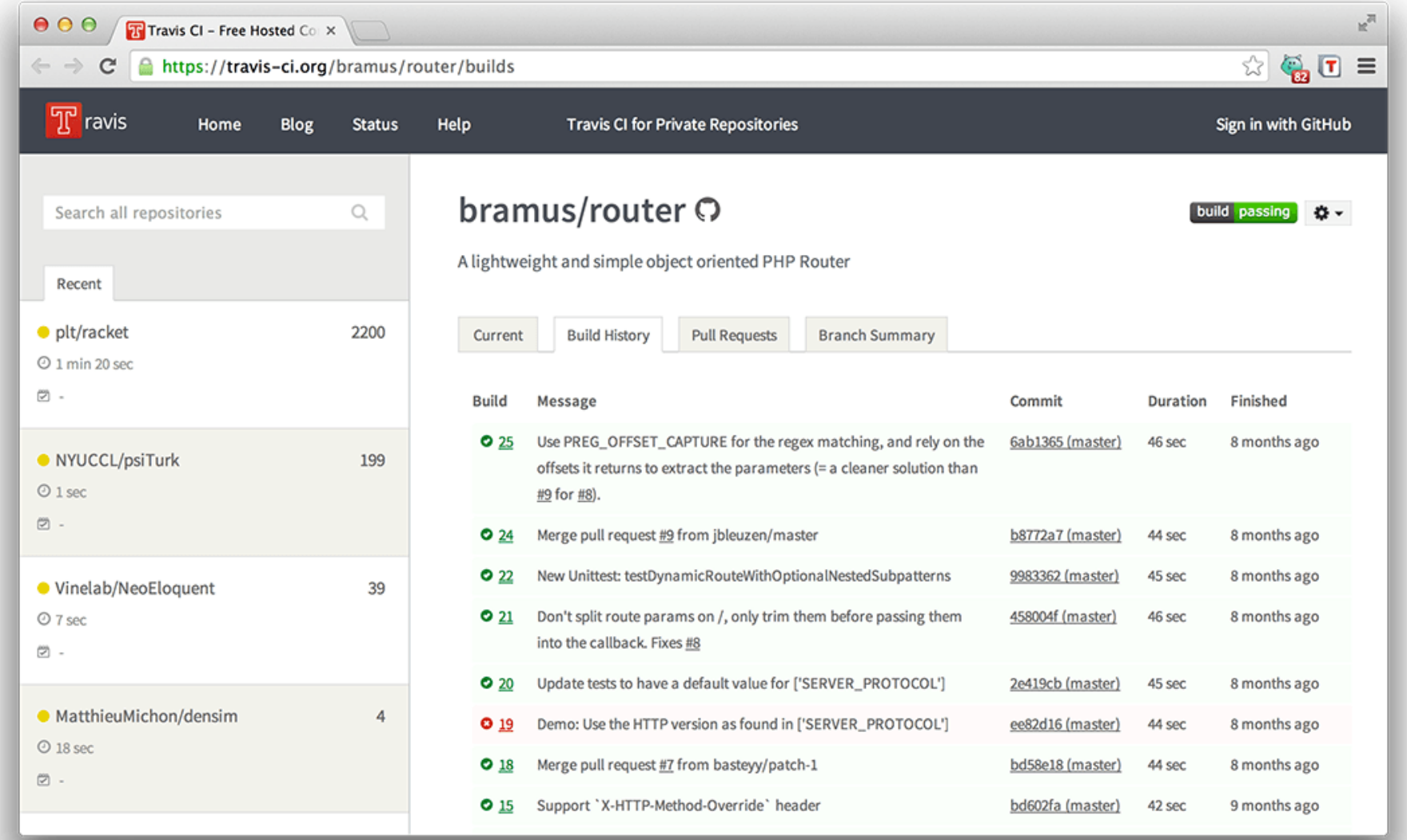

ChatGPT
Projects · JiHu GitLab
工作台 [Jenkins]
不安全 192.168.245.129:8080
重新启动即可更新
backend cpp 量化 菜鸟教程 FreeRTOS vLLM 小林coding ChatGPT AIchatOS 猪の高级大飞机 Group Replication
Jenkins
查找 (⌘+K)
cn
注销
Dashboard
新建Item
用户列表
构建历史
项目关系
检查文件指纹
Manage Jenkins
My Views
添加说明
All
S W 名称 ↓ 上次成功 上次失败 上次持续时间
cn-first 3 天 22 小时 #33 5 天 19 小时 #1 5.7 秒
cn-second 3 天 22 小时 #31 4 天 0 小时 #14 8.5 秒
图标: 小 中 大
图例
Atom feed for all
Atom feed for failures
Atom feed for just latest builds
构建队列
队列中没有构建任务
构建执行状态
built-in node (0 of 2 executors busy)
Jenkins 中文社区
REST API
Jenkins 2.440.2

Travis CI

Travis CI是一个持续集成（CI）工具，专门设计用于GitHub上的项目。它提供了一个自动化的构建和测试环境，允许开发者在代码发生变更时自动执行构建、运行测试和部署操作。

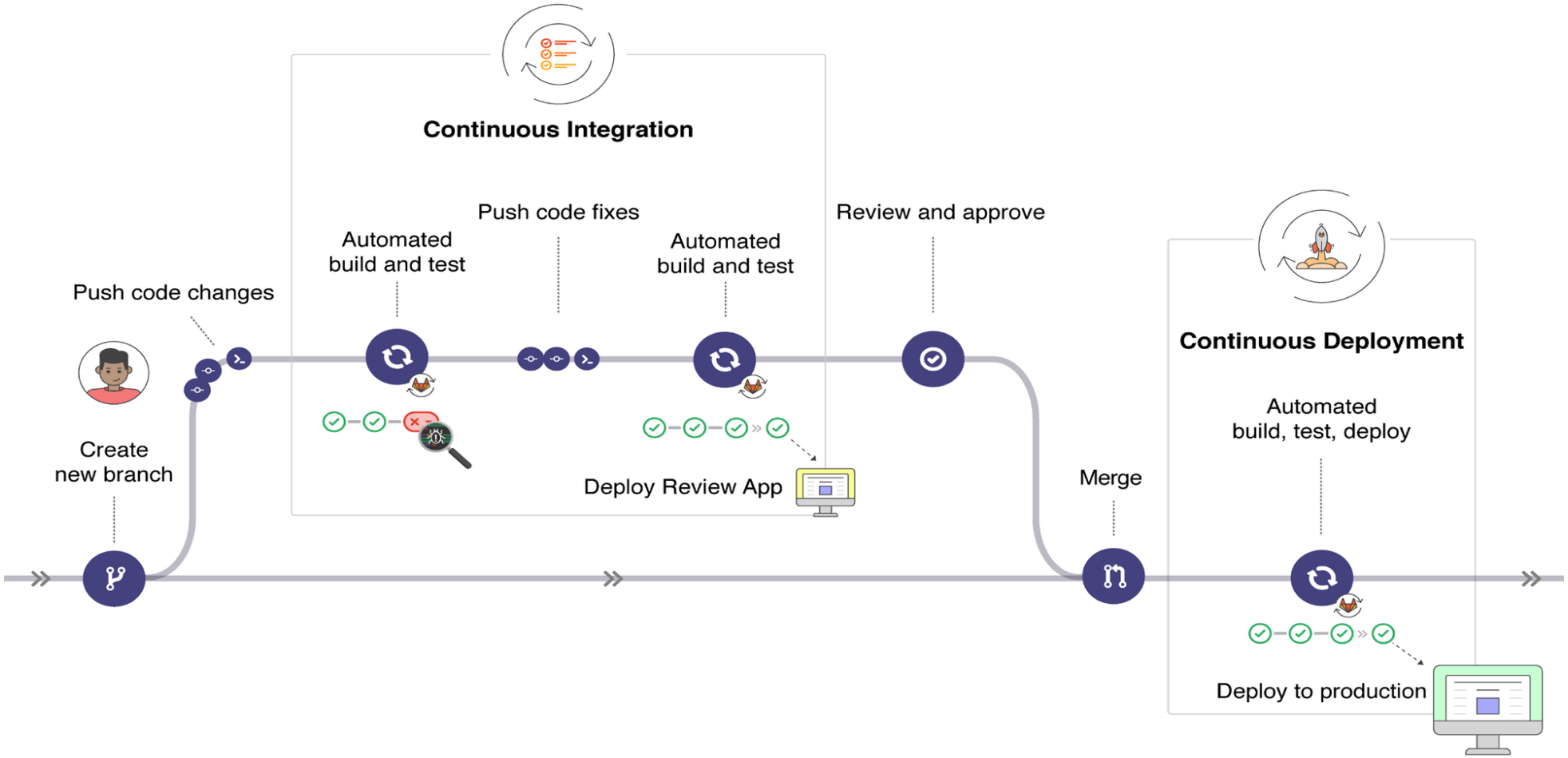


Travis CI

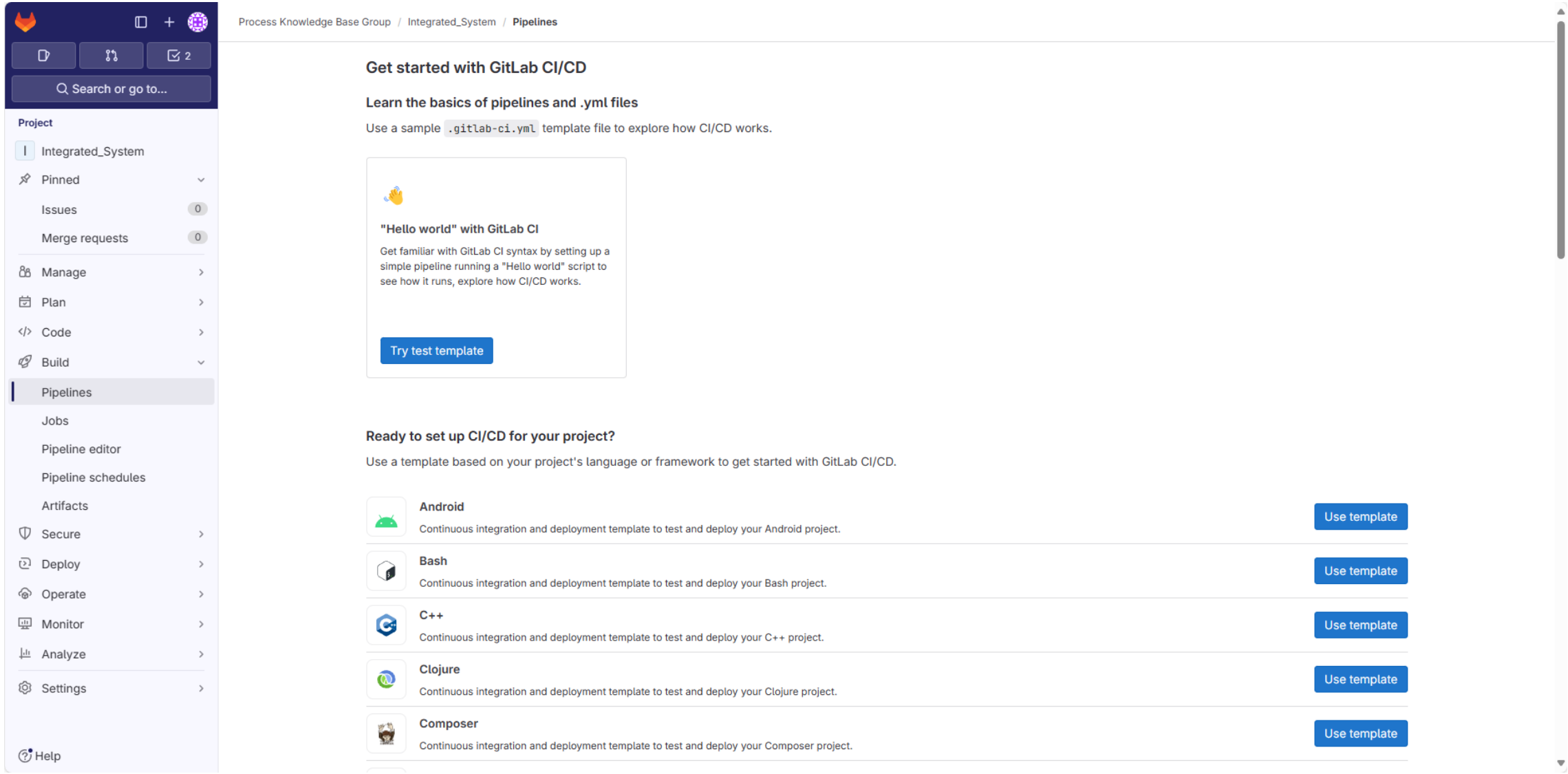

Travis CI - Free Hosted Co
https://travis-ci.org/bramus/router/builds
ravis
Home
Blog
Status
Help
Travis CI for Private Repositories
Sign in with GitHub
Search all repositories
Recent
plt/racket 2200
1 min 20 sec
NYUCCL/psiTurk 199
1 sec
Vinelab/NeoEloquent 39
7 sec
MatthieuMichon/densim 4
18 sec
bramus/router
build passing
A lightweight and simple object oriented PHP Router
Current
Build History
Pull Requests
Branch Summary
Build Message Commit Duration Finished
25 Use PREG_OFFSET_CAPTURE for the regex matching, and rely on the offsets it returns to extract the parameters (= a cleaner solution than #9 for #8). 6ab1365 (master) 46 sec 8 months ago
24 Merge pull request #9 from jbleuzen/master b8772a7 (master) 44 sec 8 months ago
22 New Unittest: testDynamicRouteWithOptionalNestedSubpatterns 9983362 (master) 45 sec 8 months ago
21 Don't split route params on /, only trim them before passing them into the callback. Fixes #8 458004f (master) 46 sec 8 months ago
20 Update tests to have a default value for ['SERVER_PROTOCOL'] 2e419cb (master) 45 sec 8 months ago
19 Demo: Use the HTTP version as found in ['SERVER_PROTOCOL'] ee82d16 (master) 44 sec 8 months ago
18 Merge pull request #7 from basteyy/patch-1 bd58e18 (master) 44 sec 8 months ago
15 Support `X-HTTP-Method-Override` header bd602fa (master) 42 sec 9 months ago

GitLab CI/CD

- GitLab CI/CD 是 GitLab 提供的集成式的持续集成和持续部署解决方案。它允许开发团队在同一个平台上管理代码仓库、构建和测试流程、以及应用部署，从而实现全面的 DevOps 流程。



GitLab CI/CD


Process Knowledge Base Group / Integrated_System / Pipelines
Search or go to...
Project
Integrated_System
Pinned
Issues 0
Merge requests 0
Manage
Plan
Code
Build
Pipelines
Jobs
Pipeline editor
Pipeline schedules
Artifacts
Secure
Deploy
Operate
Monitor
Analyze
Settings
Help
Get started with GitLab CI/CD
Learn the basics of pipelines and .yml files
Use a sample .gitlab-ci.yml template file to explore how CI/CD works.
"Hello world" with GitLab CI
Get familiar with GitLab CI syntax by setting up a simple pipeline running a "Hello world" script to see how it runs, explore how CI/CD works.
Try test template
Ready to set up CI/CD for your project?
Use a template based on your project's language or framework to get started with GitLab CI/CD.
Android
Continuous integration and deployment template to test and deploy your Android project.
Use template
Bash
Continuous integration and deployment template to test and deploy your Bash project.
Use template
C++
Continuous integration and deployment template to test and deploy your C++ project.
Use template
Clojure
Continuous integration and deployment template to test and deploy your Clojure project.
Use template
Composer
Continuous integration and deployment template to test and deploy your Composer project.
Use template

3.7 配置管理工具

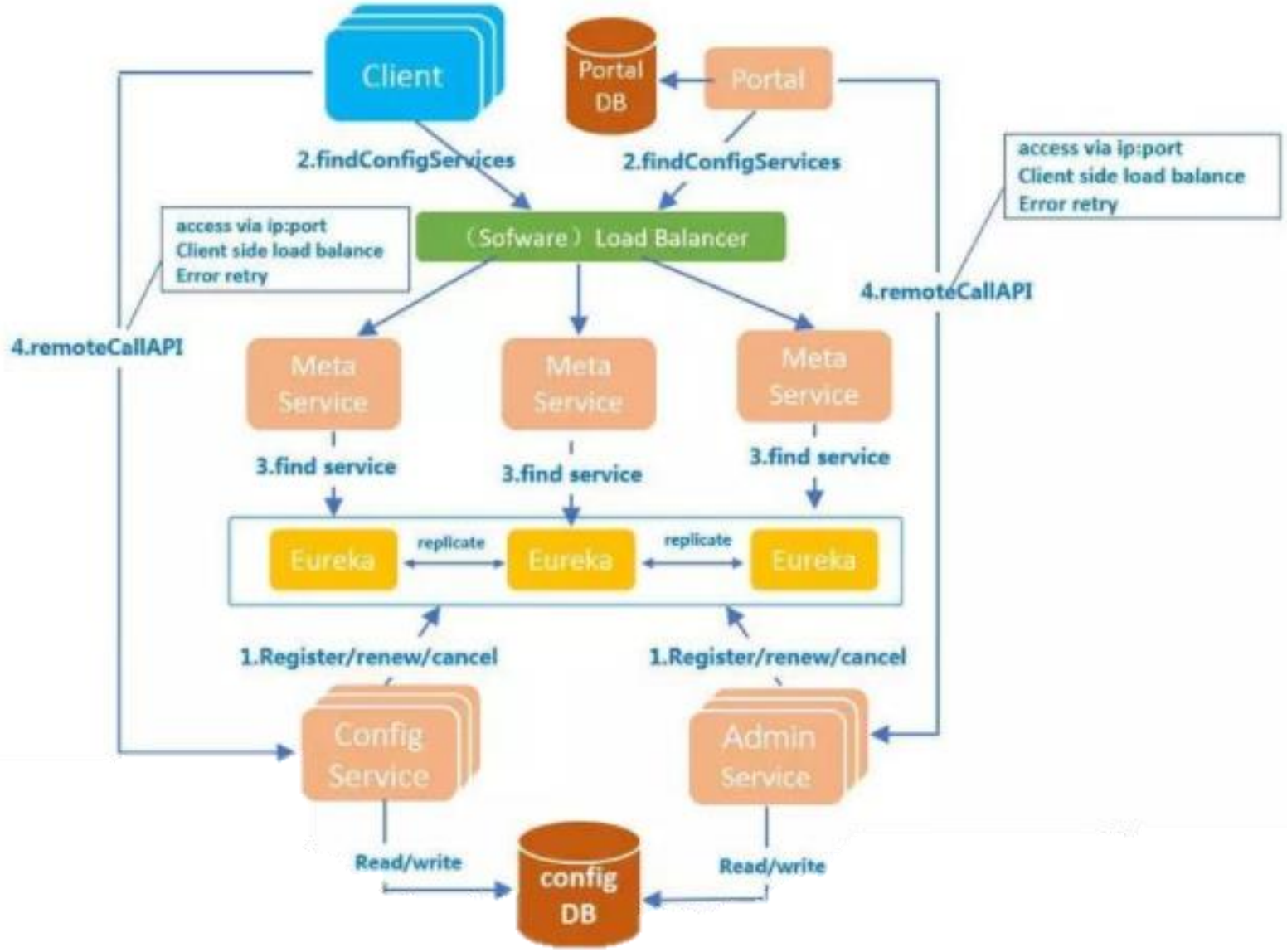
- 分布式配置中心，能够集中化管理应用的不同环境、不同集群的配置，配置修改后能够实时推送到应用端，并且具备规范的权限、流程治理等特性。
- **统一管理不同环境、不同集群的配置**
- **配置修改实时生效**
- **版本发布管理**
- **权限管理、发布审核、操作审计**
- **客户端配置信息监控**





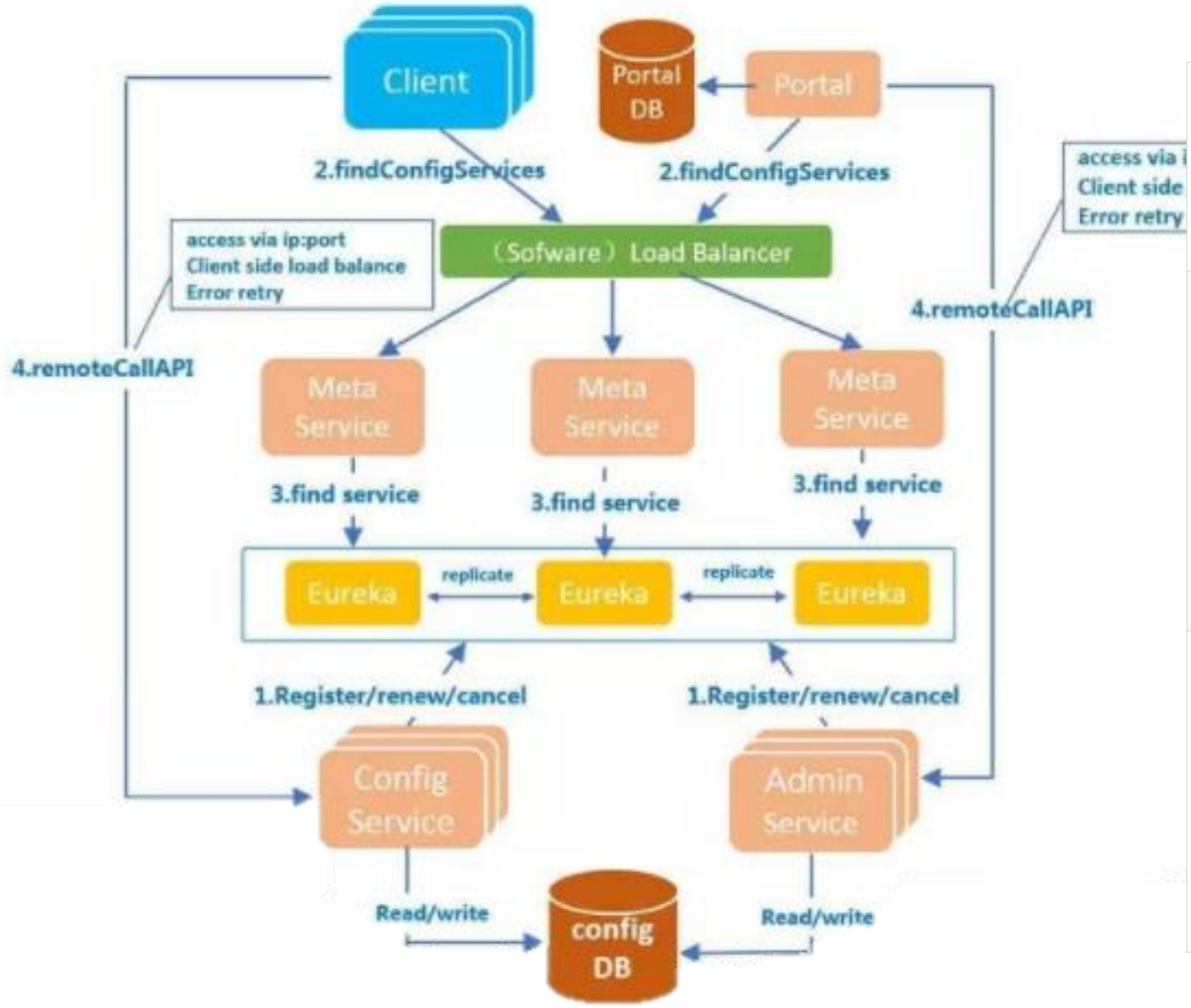


Apollo



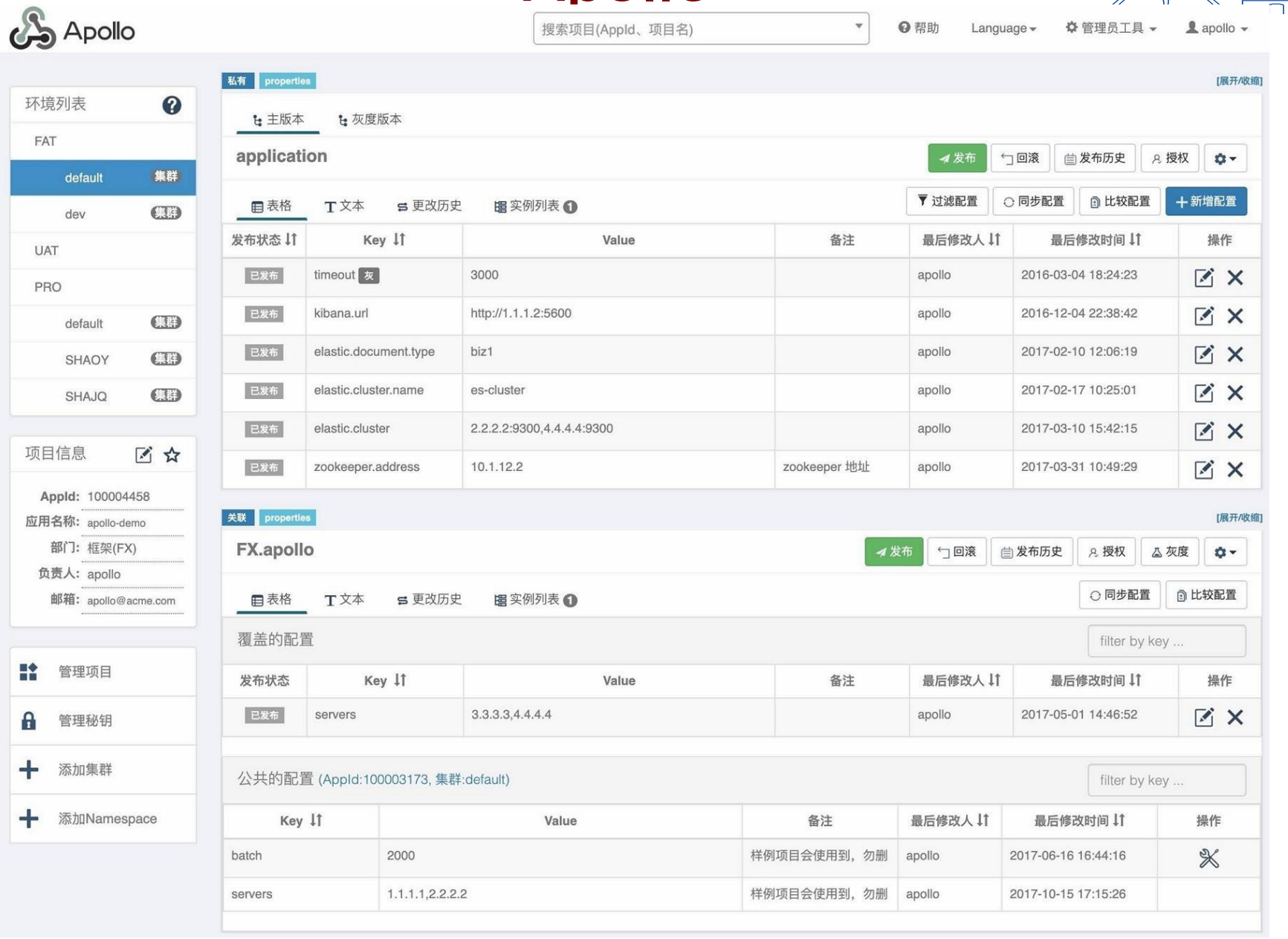
ConfigService	提供配置获取接口 提供配置推送接口 服务于Apollo客户端
AdminService	提供配置管理接口 提供配置修改发布接口 服务于管理界面Portal
Client	为应用获取配置，支持实时更新 通过MetaServer获取ConfigService的服务列表 使用客户端软负载SLB方式调用ConfigService
Portal	配置管理界面 通过MetaServer获取AdminService的服务列表 使用客户端软负载SLB方式调用AdminService

Apollo



Eureka	用于服务发现和注册 Config/AdminService注册实例并定期报心跳 和ConfigService在一起部署
MetaServer	Portal通过域名访问MetaServer获取AdminService的地址列表 Client通过域名访问MetaServer获取ConfigService的地址列表 相当于一个Eureka Proxy逻辑角色，和ConfigService在一起部署
NginxLB	和域名系统配合，协助Portal访问MetaServer获取AdminService地址列表 和域名系统配合，协助Client访问MetaServer获取ConfigService地址列表 和域名系统配合，协助用户访问Portal进行配置管理

Apollo


Apollo
搜索项目(AppId、项目名)
帮助
Language
管理员工具
apollo
环境列表
FAT
default 集群
dev 集群
UAT
PRO
default 集群
SHAOY 集群
SHAJQ 集群
项目信息
AppId: 100004458
应用名称: apollo-demo
部门: 框架(FX)
负责人: apollo
邮箱: apollo@acme.com
管理项目
管理秘钥
添加集群
添加Namespace
私有 properties
[展开/收缩]
主版本
灰度版本
application
发布
回滚
发布历史
授权
表格
文本
更改历史
实例列表 1
过滤配置
同步配置
比较配置
新增配置
发布状态
Key
Value
备注
最后修改人
最后修改时间
操作
已发布 timeout 灰 3000 apollo 2016-03-04 18:24:23
已发布 kibana.url http://1.1.1.2:5600 apollo 2016-12-04 22:38:42
已发布 elastic.document.type biz1 apollo 2017-02-10 12:06:19
已发布 elastic.cluster.name es-cluster apollo 2017-02-17 10:25:01
已发布 elastic.cluster 2.2.2.2:9300,4.4.4.4:9300 apollo 2017-03-10 15:42:15
已发布 zookeeper.address 10.1.12.2 zookeeper 地址 apollo 2017-03-31 10:49:29
关联 properties
[展开/收缩]
FX.apollo
发布
回滚
发布历史
授权
灰度
表格
文本
更改历史
实例列表 1
同步配置
比较配置
覆盖的配置
filter by key ...
发布状态
Key
Value
备注
最后修改人
最后修改时间
操作
已发布 servers 3.3.3.3,4.4.4.4 apollo 2017-05-01 14:46:52
公共的配置 (AppId:100003173, 集群:default)
filter by key ...
Key
Value
备注
最后修改人
最后修改时间
操作
batch 2000 样例项目会使用到，勿删 apollo 2017-06-16 16:44:16
servers 1.1.1.1,2.2.2.2 样例项目会使用到，勿删 apollo 2017-10-15 17:15:26

3.8 云平台

以AWS为例



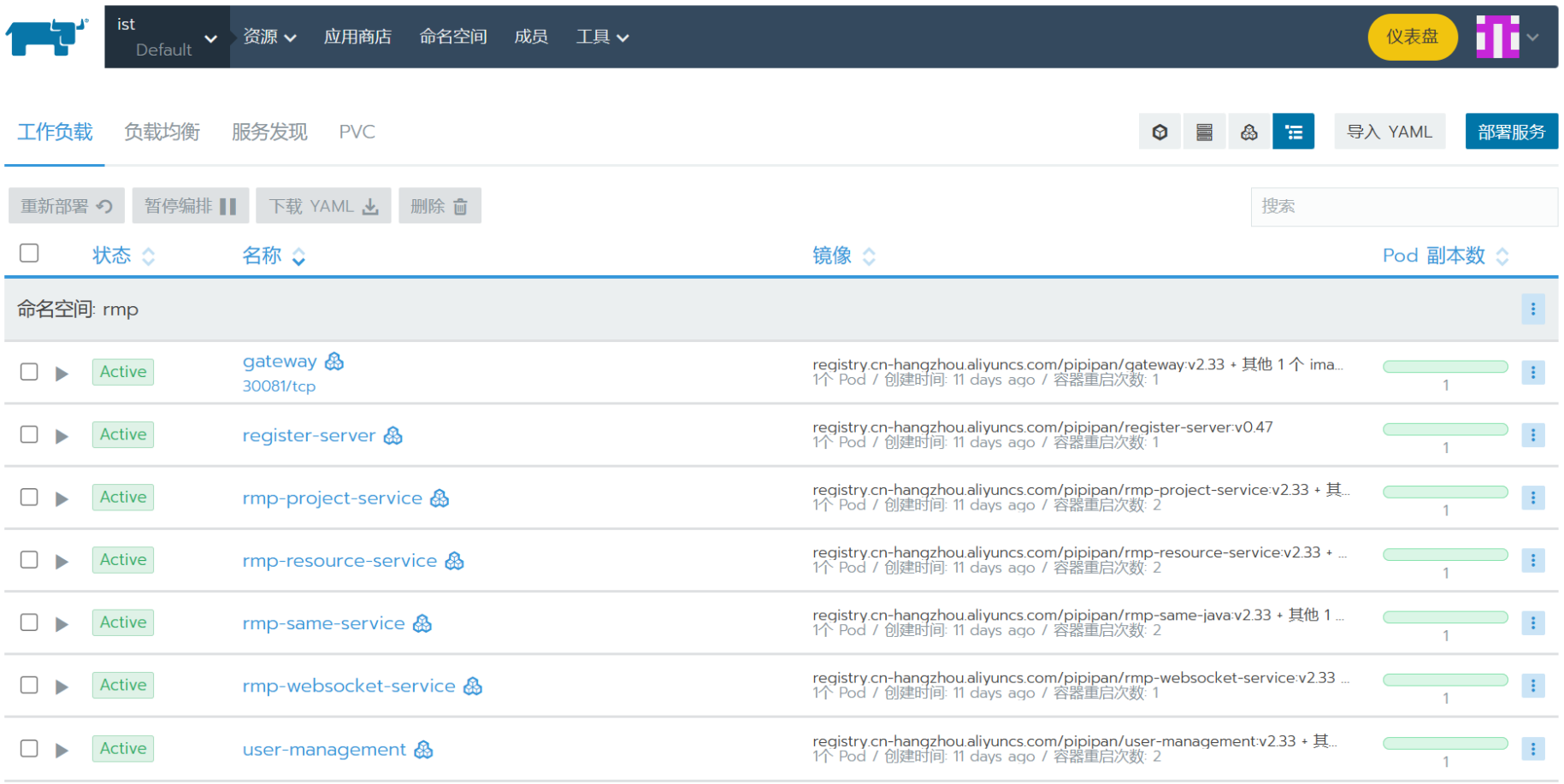
- **计算服务：**
  - EC2：可扩展的虚拟服务器。
  - Lambda：无服务器的事件驱动计算。
- **存储服务：**
  - S3：持久性对象存储。
  - EBS：持久性块存储。
- **数据库服务：**
  - RDS：托管关系型数据库。
  - DynamoDB：托管 NoSQL 数据库。
- **网络服务：**
  - VPC：虚拟私有云。
  - CloudFront：内容分发网络。

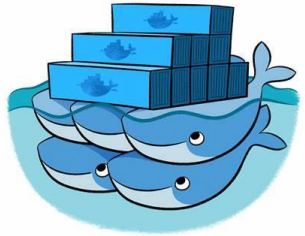




3.9 容器调度程序

- Rancher







Apache MESOS

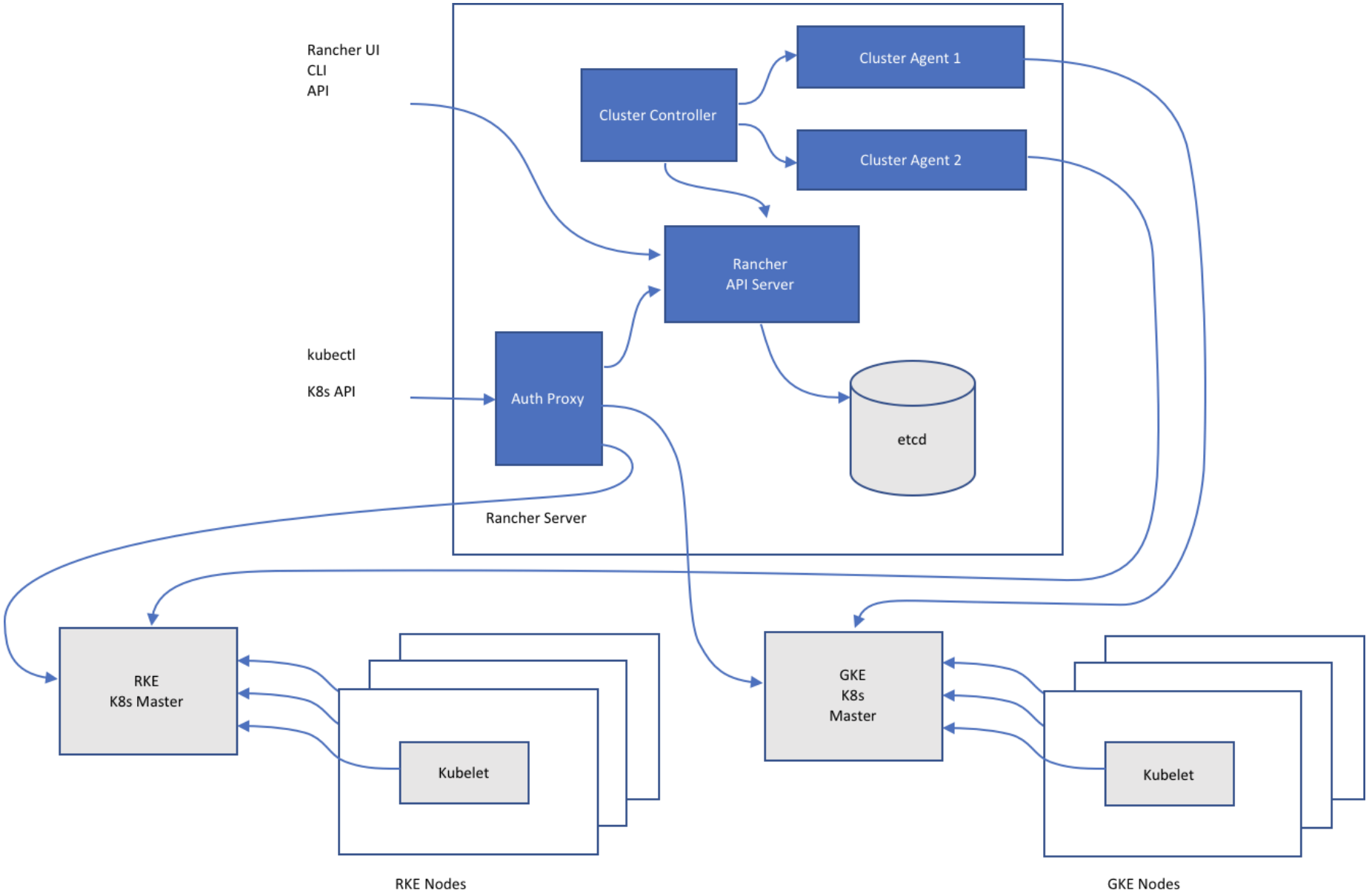


kubernetes

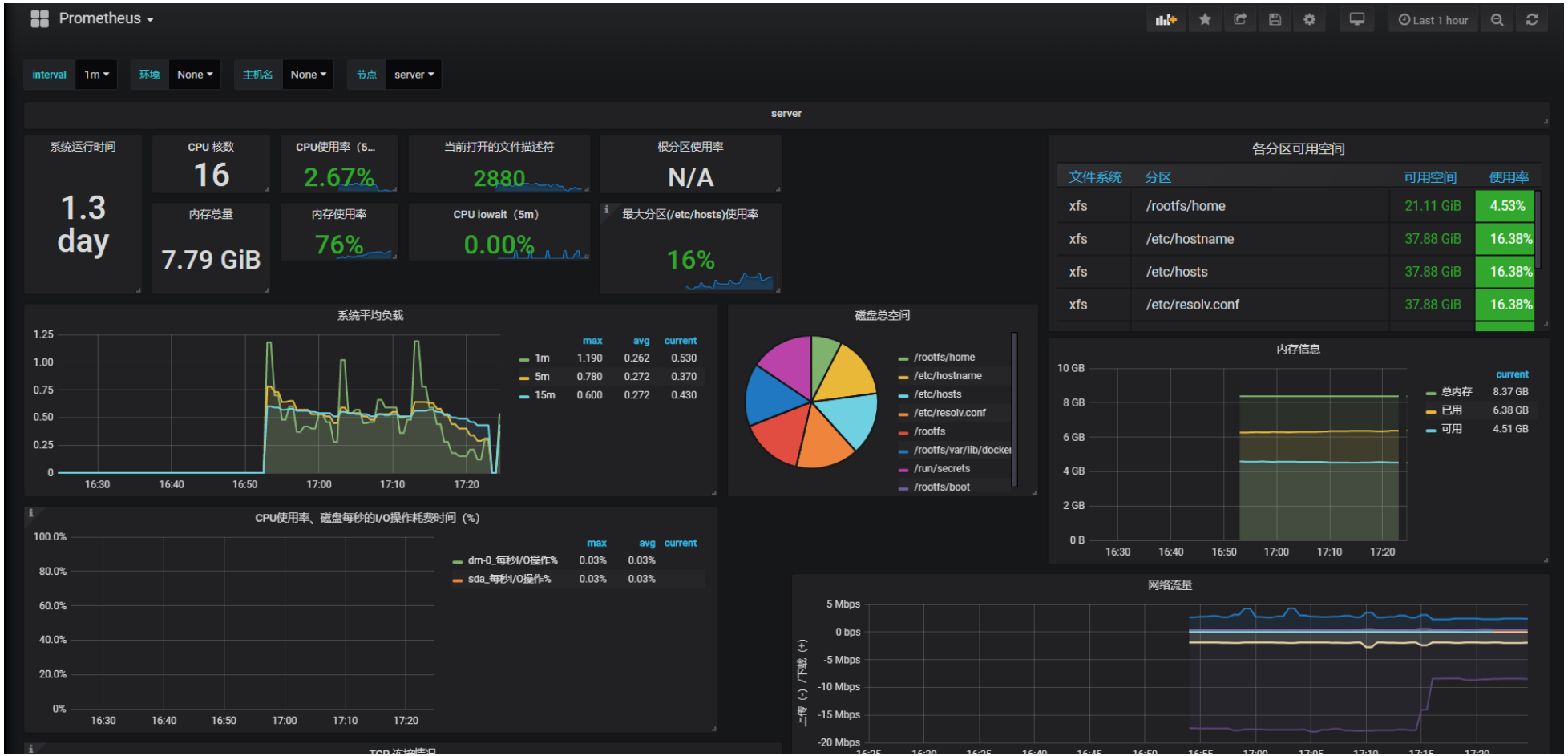
Rancher

- Rancher 可以创建来自 Kubernetes 托管服务提供商的集群，自动创建节点并安装 Kubernetes 集群，或者导入任何已经存在的 Kubernetes 集群。
- 支持集群的身份验证和基于角色的访问控制（RBAC），使系统管理员能够从一个位置控制全部集群的访问。
- 集群监控
- 容器日志转发
- 简单易用的流水线自动部署和升级。
- 多集群应用，全局 DNS，服务网格，安全扫描，集群模版和基于 OPA 的策略管理等功能。
- 总而言之，Rancher 是一个全栈式的 Kubernetes 容器管理平台，也是一个可以在任何地方都能成功运行 Kubernetes 的工具。

Rancher


Rancher UI
CLI
API
Cluster Agent 1
Cluster Controller
Cluster Agent 2
Rancher
API Server
kubectl
K8s API
Auth Proxy
etcd
Rancher Server
RKE
K8s Master
Kubelet
GKE
K8s
Master
Kubelet
RKE Nodes
GKE Nodes

3.10 监控和记录工具


Prometheus
Last 1 hour
interval 1m
环境 None
主机名 None
节点 server
server
系统运行时间
1.3 day
CPU 核数
16
CPU使用率（5...
2.67%
当前打开的文件描述符
2880
根分区使用率
N/A
内存总量
7.79 GiB
内存使用率
76%
CPU iowait（5m）
0.00%
最大分区(/etc/hosts)使用率
16%
各分区可用空间
文件系统 分区 可用空间 使用率
xfs /rootfs/home 21.11 GiB 4.53%
xfs /etc/hostname 37.88 GiB 16.38%
xfs /etc/hosts 37.88 GiB 16.38%
xfs /etc/resolv.conf 37.88 GiB 16.38%
系统平均负载
1m 1.190 0.262 0.530
5m 0.780 0.272 0.370
15m 0.600 0.272 0.430
磁盘总空间
/rootfs/home
/etc/hostname
/etc/hosts
/etc/resolv.conf
/rootfs
/rootfs/var/lib/docke
/run/secrets
/rootfs/boot
内存信息
总内存 8.37 GB
已用 6.38 GB
可用 4.51 GB
CPU使用率、磁盘每秒的I/O操作耗费时间（%）
dm-0_每秒I/O操作% 0.03% 0.03%
sda_每秒I/O操作% 0.03% 0.03%
网络流量
5 Mbps
0 bps
-5 Mbps
-10 Mbps
-15 Mbps
-20 Mbps


Stack


Skywalking

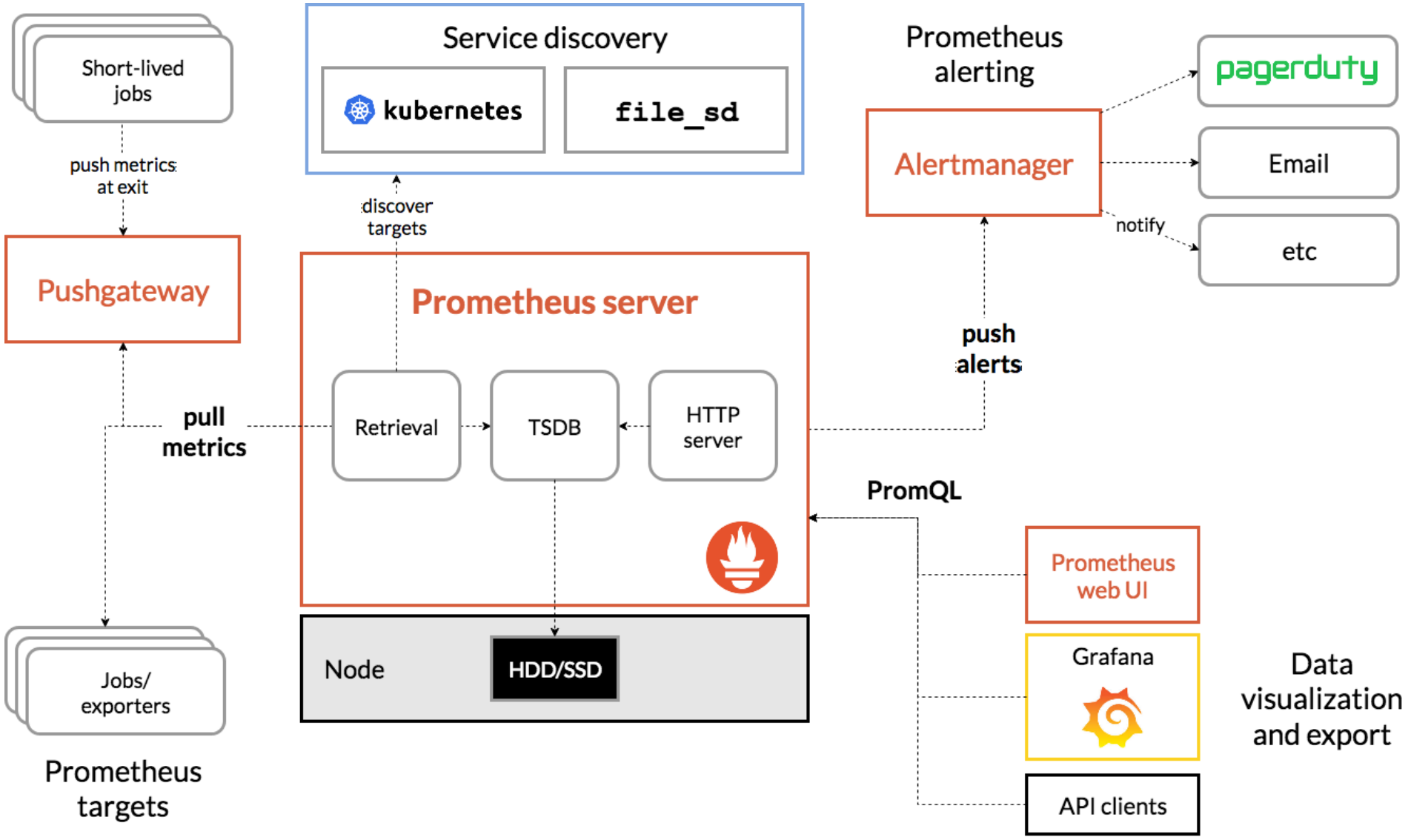


Prometheus

Prometheus 是一个开源的系统监控和警报工具包

- **多维数据模型：**
  - 支持多维度标签数据，适应动态环境。
- **强大的数据可视化：**
  - 与 Grafana 等工具集成，实现丰富的图表和仪表板
- **灵活的查询语言：**
  - 使用 PromQL 查询语言进行数据分析。
- **自动发现：**
  - 支持多种服务发现机制，如 Kubernetes等。

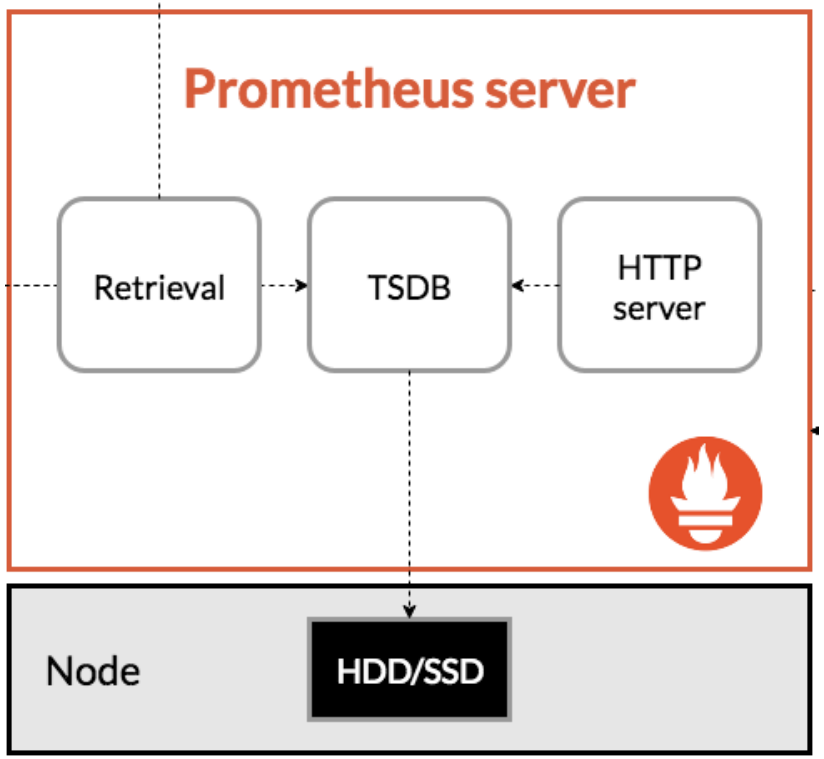
Prometheus


Short-lived jobs
push metrics at exit
Pushgateway
Service discovery
kubernetes
file_sd
discover targets
Prometheus server
Retrieval
TSDB
HTTP server
pull metrics
Node
HDD/SSD
Jobs/ exporters
Prometheus targets
Prometheus alerting
Alertmanager
pagerduty
Email
etc
notify
push alerts
PromQL
Prometheus web UI
Grafana
API clients
Data visualization and export

Prometheus

➢Prometheus Server

- 用于收集和存储时间序列数据，Prometheus 组件中的核心部分。
- 采集：可以通过静态配置管理监控目标，也可以配合使用 Service Discovery 的方式动态管理监控目标，并从这些监控目标中获取数据。
- 存储：对采集到的监控数据进行存储，Prometheus Server 本身就是一个时序数据库，将采集到的监控数据按照时间序列的方式存储在本地磁盘当中。
- 查询：对外提供了自定义的 PromQL 语言，实现对数据的查询以及分析。



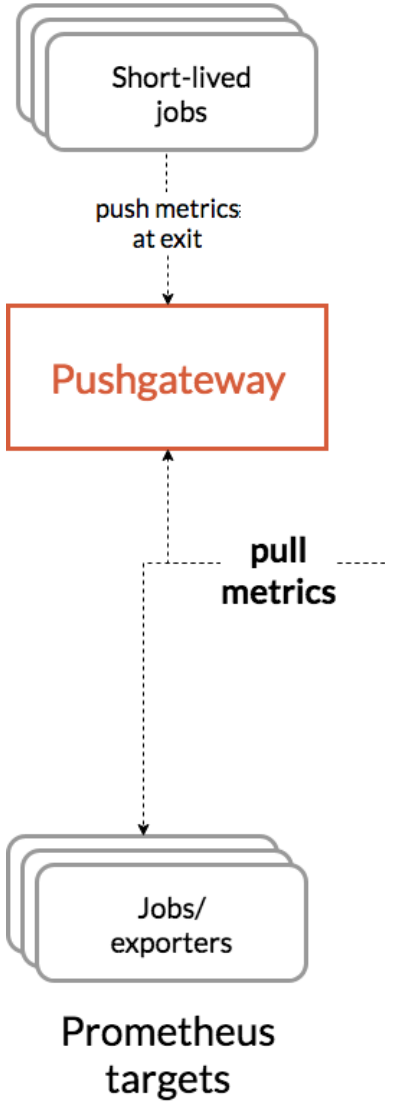
Prometheus

Exporter

- 用于暴露已有的第三方服务的 metrics 给 Prometheus。Exporter 将监控数据采集的端点通过 HTTP 服务的形式暴露给 Prometheus Server，Prometheus Server 通过访问该 Exporter 提供的 Endpoint 端点，即可获取到需要采集的监控数据。

Push Gateway

- 主要用于短期的 jobs。由于这类 jobs 存在时间较短，可能在 Prometheus 来 pull 之前就消失了。为此，这些 jobs 可以直接向 Prometheus server 端推送它们的 metrics。



Prometheus

Grafana

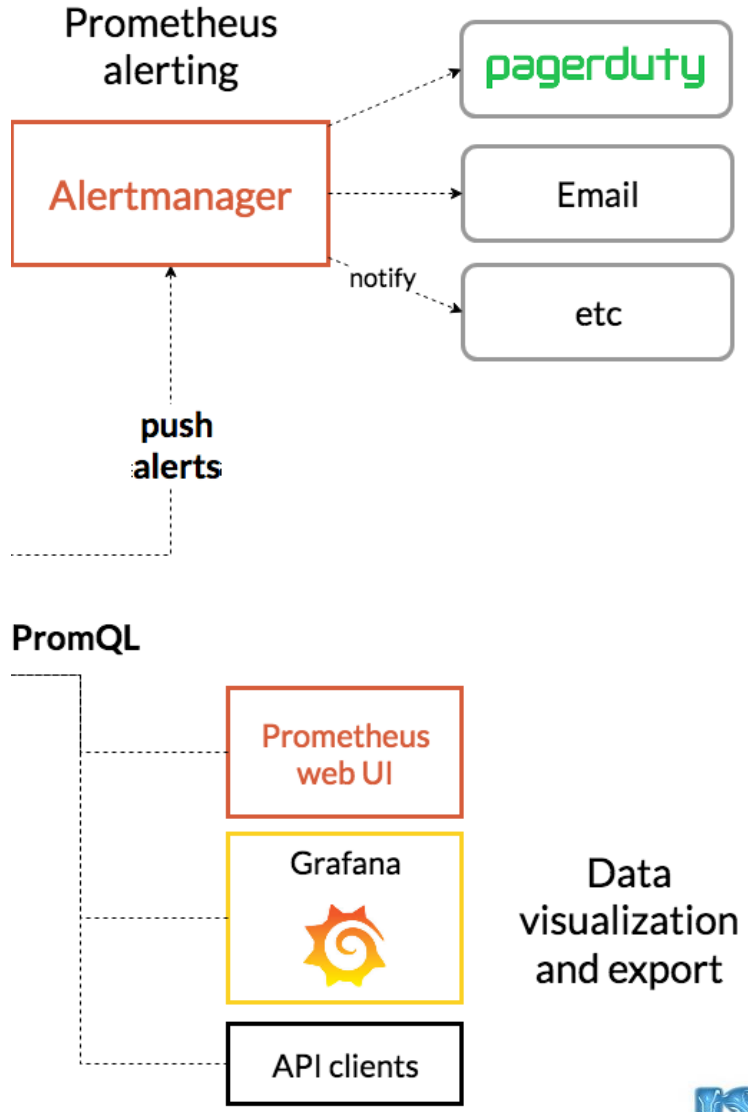
- 第三方展示工具，可以编写 PromQL 查询语句，通过 http 协议与 prometheus 集成。

AlertManager

- 从 Prometheus Server 端接收到 alerts 后，会进行去除重复数据，分组，并路由到对方的接受方式，发出报警。常见的接收方式有：电子邮件，钉钉、企业微信，pagerduty等。

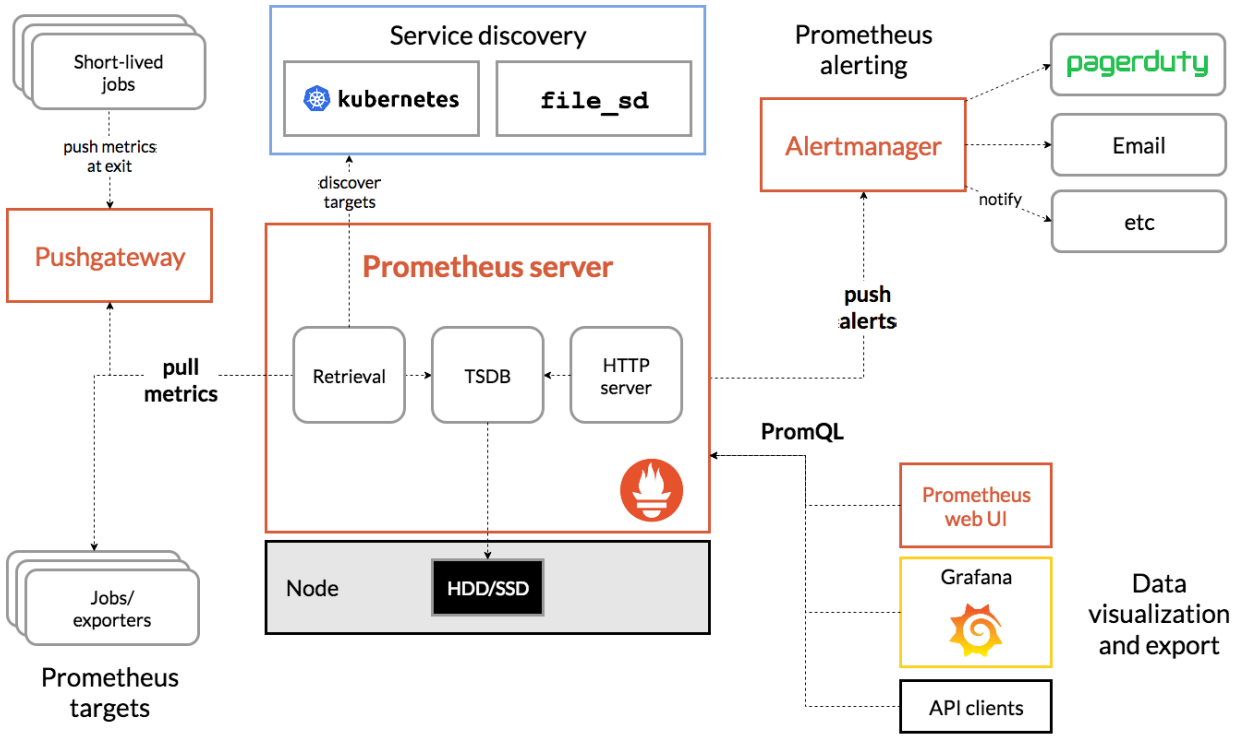
Client Library

- 客户端库，为需要监控的服务生成相应的 metrics 并暴露给 Prometheus Server。当 Prometheus Server 来 pull 时，直接返回实时状态的 metrics。



Prometheus工作流程

- 指标采集
  - prometheus server 通过 pull 形式采集监控指标，可以直接拉取监控指标，也可以通过 pushgateway 做中间环节，监控目标先 push 形式上报数据到 pushgateway；
- 指标处理
  - prometheus server 将采集的数据存储在自身 db 或者第三方 db；
- 指标展示
  - prometheus server 通过提供 http 接口，提供自带或者第三方展示系统；
- 指标告警
  - prometheus server 通过 push 告警信息到 alert-manager，alert-manager 通过"静默-抑制-整合-下发"4个阶段处理通知到观察者。



目录

- 1 微服务容器部署（Docker）
- 2 微服务容器编排（Kubernetes）
- 3 微服务系统运维（Devops）
- 4 小结

4 小结

- 阐述了软件容器化部署的实现方式
- 阐述了大规模分布式的复杂系统的容器化编排方式
- 阐述了DevOps的步骤和全生命周期工具